

A Major Project report entitled

On

Intrusion Detection System Using Raspberry Pi for IoT Devices

In partial fulfilment of the requirements for the award of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering – Cybersecurity

submitted by

Muhammed Roshiq (21E51A6233)

Ananya Krishna Murthy (21E516202)

Muhammed Wasim Pm (21E51A6234)

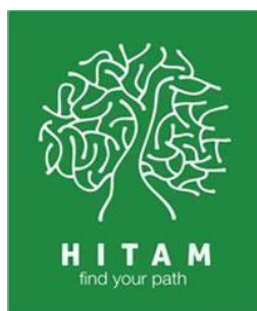
Sakshi Dubey (21E516248)

Under the Esteemed guidance of

Mr. K Ravi Kumar

Associate Professor

Department of CSE - CSC, HITAM



HYDERABAD INSTITUTE OF TECHNOLOGY AND MANAGEMENT

Autonomous Affiliated to JNTUH, Approved by AICTE, Accredited by NAAC A+, NBA.

Basuragadi (Village), Medchal (Mandal), Medchal – Malkajgiri (Dist.), Hyderabad, TS- 501401.

2024–2025

HYDERABAD INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(Autonomous, Affiliated to JNTUH, Approved by AICTE, Accredited by NAAC A+, NBA – TS 501401)

DEPARTMENT OF EMERGING TECHNOLOGY



CERTIFICATE

This is to certify that the Major project work entitled “**INTRUSION DETECTION SYSTEM USING RASPBERRY PI FOR IOT DEVICES**” is a bonafide work carried out by **MUHAMMED ROSHIQ** bearing **Roll No.(21E51A6233)**, **ANANYA KRISHNA MURTHY** bearing **Roll No.(21E51A6202)**, **MUHAMMED WASIM PM** bearing **Roll No. (21E51A6234)**, **SAKSHI DUBEY** bearing **Roll No. (21E51A6248)** in partial fulfilment of the requirements for the degree BACHELOR OF TECHNOLOGY in CSE – CYBERSECURITY during the academic year 2024-2025. The matter contained in this document has not been submitted to any other University or institute for the award of any degree.

Internal Guide

Mr. K Ravi Kumar
Associate Professor
Department of ET

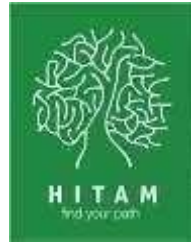
Head of the Department

Dr. M.V.A NAIDU
Professor
Department of ET

HYDERABAD INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(Autonomous, Affiliated to JNTUH, Approved by AICTE, Accredited by NAAC A+, NBA – TS 501401)

DEPARTMENT OF EMERGING TECHNOLOGY



DECLARATION

We “MUHAMMED ROSHIQ, ANANYA KRISHNA MURTHY, MUHAMMED WASIM PM, SAKSHI DUBEY” student of ‘Bachelor of Technology in CSE - CYBERSECURITY’, session: 2024-2025, Hyderabad Institute of Technology and Management, Gowdavalli, Hyderabad, Telangana State, hereby declare that the work presented in this Major project work entitled **‘INTRUSION DETECTION SYSTEM USING RASPBERRY PI FOR IOT DEVICES’** is the outcome of our own bonafide work in eighth semester and is correct to the best of our knowledge and this work has been undertaken taking care of engineering ethics. It contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

21E51A6233 MUHAMMED ROSHIQ

21E51A6202 ANANYA KRISHNA MURTHY

21E51A6233 MUHAMMED WASIM PM

21E51A6248 SAKSHI DUBEY

ACKNOWLEDGEMENT

An endeavour of a long period can be successful only with the advice of many well-wishers.

We would like to thank our chairman Sri **Mr. ARUTLA PRASHANTH** for providing all the facilities to carry the Major project work successfully.

We would like to thank our Principal **Dr. S. ARVIND** who has inspired us a lot through their speeches and provided this opportunity to carry out our Major project work successfully.

We are very thankful to our Head of the Department, **Dr. M.V.A NAIDU** and B. Tech Major Project Coordinator **Mr. BOBBY**

We would like to specially thank my internal guide **Mr. K Ravi Kumar** for technical guidance, constant encouragement, and enormous support provided to us for carrying out our Major project work.

We wish to convey our gratitude and express sincere thanks to all **D.C (Departmental Committee)** and **P.R.C (Project Review Committee)** members, non- teaching staff for their support and Cooperation rendered for successful submission of our Major project.

We also want to express our sincere gratitude to all my family members and my friends for their individual care and everlasting moral support.

21E51A6233 MUHAMMED ROSHIQ

21E51A6202 ANANYA KRISHNA MURTHY

21E51A6234 MUHAMMED WASIM PM

21E51A6248 SAKSHI DUBEY

CONTENTS

Chapter No.	Name	Page No.
	Abstract	1
1	Introduction	2
2	Literature survey	6
3	Problem definition and Methodologies	10
4	System Methodologies	14
5	Implementation	19
6	Result	47
7	Conclusion and Future enhancement	49
	References	50
	Paper publication copy	

List of Figures

FIGURENO	FIGURENAME	PAGENO
4.1	Snort Rule Explanation	15
5.1	Raspberry Pi Imager	19
5.2	<i>Choosing a Device</i>	20
5.3	Choosing an OS	20
5.4	Storage Devices	21
5.5	OS Customization Option	21
5.6	Writing an Image	22
5.7	Ready to boot	23
5.8	Snort Installation	23
5.9	Snort Configuration	24
5.10	RPI Placement	25
5.11	hping3 installation	26
5.12	hping3 Attack	26
5.13	Nmap installation	27
5.14	Nmap Attack	27
5.15	Ettercap installation	28
5.16	Ettercap attack	28
5.17	Hydra installation	29
5.18	Hydra Attack	29
5.19	Python Installation	30
5.20	Python script	31
5.21	Running a script	32
5.22	Alert Notification in Telegram	32
5.23	Local rules	35
5.24	Attack Detection	36
5.25	Camera Module integration	37
5.26	flask server	37
5.27	Authentication page	38
5.28	Streaming camera	38
5.29	Alert	39

6.1	RPi with Camera module	47
6.2	Snort IDS Log File	47
6.3	Python telegram alert script	48
6.4	Alert message	48

LIST OF ABBREVIATIONS

RPI	Raspberry Pi
IDS	Intrusion Detection System
IoT	Internet of Things
DoS	Denial of Service
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
HTTP	Hypertext Transfer Protocol
ICMP	Internet Control Message Protocol
FTP	File Transfer Protocol
API	Application Programming Interface
OS	Operating System
LAN	Local Area Network
CPU	Central Processing Unit
RAM	Random Access Memory
SSL	Secure Sockets Layer
SSH	Secure Shell
Bot	Robot (in the context of a Telegram bot)
SPAN	Switch Port Analyzer

ABSTRACT

The escalating proliferation of Internet of Things (IoT) devices accentuates the urgency for robust cybersecurity solutions. The objective is to enhance IoT security by merging Snort IDS features with the efficiency and compactness of Raspberry Pi (RPI).

The proposal advocates for a tailored security solution that is not only lightweight but also cost-effective, catering to the resource constraints typical of IoT devices.

It involves an exhaustive evaluation of Snort IDS performance on the RPi across various cyber threat scenarios, including ICMP, SYN, and HTTP floods, brute force attempts, Nmap scans, and ARP spoofing. The findings underscore the system's adeptness in promptly identifying and responding to these threats, demonstrating its viability for safeguarding IoT ecosystems.

A distinctive facet of the proposal is the seamless integration of real-time alerting mechanisms using Telegram. This not only enhances the responsiveness of the security system but also provides an accessible and user-friendly means of staying informed about potential cyber threats

CHAPTER 1

INTRODUCTION

1.1. OVERVIEW

The rapid expansion of the Internet of Things (IoT) necessitates robust cybersecurity solutions. This proposal proposes an innovative integration of the Snort Intrusion Detection System (IDS) with the Raspberry Pi (RPI) platform to address security challenges in IoT ecosystems. Leveraging the lightweight and cost-effective nature of the RPI, the proposal aims to fortify IoT environments against diverse cyber threats.

The key objective is the configuration and thorough testing of the Snort IDS on the RPI platform. Performance evaluations will focus on detecting and mitigating common cyber threats in IoT scenarios, including ICMP, SYN, and HTTP floods, brute force attempts, Nmap scans, and ARP spoofing.

A distinctive feature involves real-time alerting through Telegram integration, providing instant notifications in response to detected threats. This enhances incident response speed and engages users or administrators in the security dynamics of their IoT network.

1.2. PROJECT DESCRIPTION

Strategic integration of the Snort Intrusion Detection System (IDS) with the Raspberry Pi (RPI) platform to enhance cybersecurity within the Internet of Things (IoT). By incorporating Snort IDS, a widely acclaimed open-source security tool, into lightweight and cost-effective RPI devices, the proposal aims to proactively address cybersecurity challenges prevalent in resource-constrained IoT ecosystems.²

To establish a nimble and resource-efficient security solution, the proposal includes a thorough evaluation of Snort IDS's performance. The focus is on assessing its capabilities in detecting and mitigating diverse cyber threats commonly encountered in IoT scenarios, covering prevalent attack vectors such as ICMP, SYN, and HTTP floods, brute force attempts, Nmap scans, and ARP spoofing.

A distinctive feature of this approach is the integration of a real-time alerting mechanism using the Telegram platform. This ensures prompt notifications upon the detection

of potential cyber threats, facilitating swift incident response and fostering heightened situational awareness.

1.3. OBJECTIVE

The primary objective of this proposal is to create a robust cybersecurity solution tailored for Internet of Things (IoT) environments by integrating the Snort Intrusion Detection System (IDS) with the Raspberry Pi (RPI). This integration aims to provide a lightweight, cost-effective security mechanism for IoT devices, addressing the prevalent challenges of inadequate security measures in the rapidly proliferating landscape of IoT devices. The proposal involves a comprehensive evaluation of Snort IDS's performance in identifying and mitigating various cyber threats commonly encountered in IoT ecosystems. The assessed threats include ICMP, SYN, and HTTP floods, brute force attempts, Nmap scans, and ARP spoofing. Additionally, the proposal incorporates a real-time alerting mechanism through Telegram integration, ensuring prompt notifications upon the detection of potential cyber threats. The ultimate goal is to contribute to the development of a resource-efficient and scalable security solution, enhancing the overall resilience and security posture of IoT ecosystems.

1.4. ABOUT THE PROJECT

This proposal represents a pioneering effort to confront the intricate cybersecurity challenges inherent in the expansive domain of the Internet of Things (IoT). Through the amalgamation of the Snort Intrusion Detection System (IDS) with the Raspberry Pi (RPI), we endeavor to craft a bespoke, nimble, and economical security solution meticulously tailored for IoT devices. The contemporary proliferation of IoT devices, bereft of robust cybersecurity measures, accentuates their susceptibility to an array of cyber threats. Our undertaking involves a rigorous exploration of Snort IDS's efficacy in identifying and thwarting diverse cyber threats commonly encountered in IoT environments, encompassing ICMP, SYN, and HTTP floods, brute force attempts, Nmap scans, and ARP spoofing.

Beyond the realm of threat detection, our proposal introduces an innovative facet by seamlessly integrating Telegram for real-time alerting. This feature ensures expeditious notifications upon the identification of potential cyber threats, facilitating swift responses and mitigation efforts. With a comprehensive evaluation of Snort IDS's functionalities, our objective is to contribute to the development of a resource-efficient, scalable security solution.

The proposal aspires to elevate the overall security resilience of IoT ecosystems, providing a robust defense mechanism against the ever-evolving landscape of cyber threats. The successful integration of Telegram enhances the proposal's practicality, allowing users to receive instant alerts and respond promptly to potential security breaches.

1.5. ORGANIZATION OF THE PROPOSAL

- **Chapter 1: Introduction**

This chapter serves as an introduction to the proposal, offering a comprehensive overview of its objectives, description, and underlying motives. It sets the stage for the subsequent chapters by outlining the key aspects and goals of the proposal.

- **Chapter 2: Literature Survey**

This chapter presents an extensive literature survey, delving into various techniques relevant to the proposal. Drawing from technical papers published in IEEE conferences and journals, it provides a comprehensive overview of existing research and developments in the field, offering valuable insights for the proposed work.

- **Chapter 3: Problem Definition and Methodologies**

In this chapter, the problem definition and methodologies are thoroughly discussed. It lays the groundwork for the proposed solution by identifying the key challenges and outlining the methodologies that will be employed to address them. This chapter serves as a crucial foundation for the subsequent design and implementation phases.

- **Chapter 4: System Methodologies**

Focused on the design process and system requirements, this chapter provides detailed insights into the architectural considerations and functional specifications of the proposed solution. It outlines the design principles and requirements that will guide the development process, ensuring alignment with the project objectives.

- **Chapter 5: Implementation**

The implementation chapter offers a practical demonstration of the proposal, showcasing how the theoretical concepts discussed in earlier chapters are translated into real-world applications. It provides a detailed overview of the implementation steps, methodologies, and technologies employed to realize the proposed solution.

- **Chapter 6: Result**

This chapter outlines the results obtained from testing the implemented system in real-time scenarios. It highlights the system's effectiveness in detecting IoT security threats, analyzing its performance, detection accuracy, and alert responsiveness. The findings validate the practicality and reliability of the proposed solution.

- **Chapter 7: Conclusion & Future Enhancement**

Serving as the conclusion of the document, this chapter offers a comprehensive summary of the proposal, highlighting key findings, insights, and contributions. It reflects on the journey undertaken throughout the document, reiterating the significance of the proposed work and its potential implications for future research and practice.

CHAPTER 2

LITERATURE SURVEY

1.Sherasiya,S.A.,etal.(2025)

Proposed a real-time anomaly-based IDS for IoT networks utilizing Long Short-Term Memory (LSTM) deep learning algorithms. Implemented on Raspberry Pi, the system demonstrated high detection accuracy with low false positives. It utilized the NSL-KDD dataset to train and validate its model, emphasizing the use of lightweight neural networks optimized for edge devices. The system was tested under various network conditions and attack types including DoS, R2L, and Probe, confirming its adaptability and robustness in smart environments.

2.Tabrez,S.,etal.(2025)

Developed a rule-based IDS using Snort on Raspberry Pi 3, tailored for IoT environments. The system effectively detected unauthorized access, SYN floods, port scans, and brute-force login attempts. By integrating real-time email alerts and a web dashboard for log analysis, the system empowered network administrators to respond promptly to threats. The study also explored the use of dynamic IP blocking based on repeated attack attempts to harden network defenses.

3.Rashid,Q.Y.,etal.(2025)

Introduced a hybrid IDS combining Decision Tree, K-Nearest Neighbor, and Random Forest classifiers, implemented on Raspberry Pi. This ensemble approach improved accuracy and reduced false positives compared to single-model systems. The authors also proposed a data pre-processing pipeline that normalized and filtered traffic features, enhancing model training efficiency. Real-world network traffic was simulated to evaluate the model's generalization capability across different IoT use cases.

4.Jonnalagadda,S.,etal.(2024)

Developed a smart home security framework integrating Snort IDS with Raspberry Pi for real-time intrusion detection. The system captured, filtered, and analyzed network traffic to detect abnormal patterns and generated alerts for activities like port scans and DoS attempts. It utilized an SMS alert system and a web-based UI for administrators to monitor events. The authors proposed future upgrades such as biometric user authentication and cloud-based data backup.

5.Manogna,G.,etal.(2024)

Presented a Raspberry Pi-based IDS using Decision Tree and K-Nearest Neighbors to identify

anomalous behavior in IoT traffic. The system, trained on the NSL-KDD dataset, prioritized efficient computation on limited hardware. Testing showed that Raspberry Pi could handle real-time packet processing without significant latency. The study concluded that edge computing with AI-based intrusion detection could significantly reduce cloud dependency and improve privacy.

6.Kumar,M.,etal.(2025)

Proposed a modular and scalable IDS framework using Snort and Raspberry Pi for securing educational institutes and smart buildings. The system featured layered security with distributed sensors across various departments. Custom Snort rules tailored for campus-specific threats (like student device infections and peer-to-peer abuse) enhanced detection. The architecture allowed centralized log aggregation and response from a main server.

7.Jyothirmmai,M.,etal.(2024)

Implemented an SVM-based IDS on Raspberry Pi to detect and mitigate DoS and R2L attacks in IoT networks. A feature reduction technique (PCA) was used to minimize computational load and enhance performance. Raspberry Pi's GPIO pins were also utilized to trigger alarms or indicators upon threat detection. The authors emphasized extending the system to cover mobile IoT devices and wearable tech.

8.Rizvi,S.,etal.(2024)

Presented a dual-layered IDS combining Snort with anomaly detection using K-means clustering, deployed on Raspberry Pi. The system monitored traffic patterns, segregated normal from suspicious behavior, and classified unknown threats. It featured a central logging system accessible via web dashboard, and periodic model retraining for adaptive learning. This hybrid model significantly improved zero-day threat detection compared to traditional Snort-based systems.

9.Asad,M.,etal.(2025)

Focused on optimizing Snort rules for Raspberry Pi by analyzing rule performance and redundancy. The study proposed automated scripts to evaluate rule conflicts and prioritize high-impact rules. It also explored the integration of GPU modules with Raspberry Pi to enhance packet processing speed. Performance benchmarks indicated up to 30% increase in throughput after optimization.

10.Debnath,S.,etal.(2024)

Designed a distributed IDS architecture using multiple Raspberry Pis in a mesh network. Each

Pi handled localized monitoring, while a master node aggregated and visualized traffic logs. Implemented in a smart office setup, the system demonstrated high scalability, minimal delay in threat detection, and efficient load distribution. It also included a fallback mechanism to ensure continuity during node failure.

11.Garalov,T.,&El-hajj,M.(2024)

Evaluated the performance of Raspberry Pi-based IDS in detecting network threats targeting IoT devices. The system was tested in a controlled lab simulating real-world smart home scenarios. Attacks like ARP spoofing, TCP floods, and unauthorized device access were successfully detected. The research provided a comparative analysis with commercial IDS solutions, showing comparable results at a fraction of the cost.

12.Saha,G.(2025)

Created a camera-based home IDS using Raspberry Pi and Firebase. The system triggered image capture using PIR sensors, stored frames in Firebase, and sent alerts to a mobile application. This setup allowed cloud-based monitoring from remote locations. The study explored enhancements like video analytics and biometric authentication to improve context-aware intrusion detection.

13.Mane,T.,etal.(2024)

Designed a wildlife intrusion detection system using Raspberry Pi to safeguard agricultural fields. The system combined thermal imaging, PIR sensors, and animal sound deterrents. AI algorithms processed motion data to identify specific species and activated deterrents accordingly. The system offered real-time SMS alerts and a mobile dashboard for farmers, reducing crop losses.

14.B3TA-BLOCKER(2024)

Developed a machine learning-powered IDS using LSTM networks deployed on Raspberry Pi. Trained with the CSE-CIC-IDS 2018 dataset, the system identified sophisticated attack types like botnets and infiltration attempts. It achieved over 95% accuracy in lab testing. A secondary feature allowed automatic firewall updates to block IPs of identified attackers.

15.Roberts,D.(2024)

Built an IDS using Snort and Tshark on Raspberry Pi with a focus on educational training. The setup included an LCD interface for rule management, email alert system, and a graphical visualization dashboard using Grafana. Custom packet capture filters improved the system's

sensitivity to specific threat types. The project demonstrated the viability of DIY IDS solutions for cybersecurity education and small enterprise use.

CHAPTER 3

PROBLEM DEFINITION AND METHODOLOGIES

3.1 PROBLEM DEFINITION

The rapid proliferation of IoT devices lacks robust cybersecurity, leaving them vulnerable to cyber threats. IoT environments require a lightweight, cost-effective, and scalable security solution to protect devices from cyber threats. Traditional security solutions for Internet of Things (IoT) devices often use up a lot of resources, energy, and bandwidth. A cost-effective, all-in-one solution for modern cybersecurity is needed.

3.2 EXISTING SYSTEM

Traditional security solutions for IoT devices, including cryptographic protocols, secure routing policies, anti-malware solutions, and trust management systems, have inherent limitations. These solutions often consume excessive resources, energy, and bandwidth, making them unsuitable for resource-constrained IoT environments. Additionally, they may lack scalability and flexibility to adapt to evolving threats and device configurations.

Blockchain technology has been proposed as a modern security solution for IoT devices, it presents its own set of challenges. Issues with anonymity, scalability, and performance rates hinder its effectiveness in IoT security applications. Blockchain's decentralized nature and consensus mechanisms may introduce latency and overhead, making it impractical for real-time security enforcement in dynamic IoT environments.

3.3 PROPOSED SYSTEM

The proposed system entails the seamless integration of the Snort Intrusion Detection System (IDS) with the Raspberry Pi (RPi) platform, forming a robust cybersecurity solution tailored for Internet of Things (IoT) environments. Leveraging the capabilities of Snort IDS, renowned for its effectiveness in detecting and mitigating cyber threats, the system aims to fortify IoT ecosystems against diverse attack vectors. The lightweight and cost-effective nature of the RPi platform makes it an ideal choice for hosting Snort IDS, ensuring optimal resource utilization while maintaining robust security protocols. Through meticulous configuration and testing, the proposed system will be fine-tuned to effectively detect and respond to various cyber threats prevalent in IoT scenarios, including ICMP, SYN, and HTTP floods, brute force

attempts, Nmap scans, and ARP spoofing. Additionally, the integration of real-time alerting mechanisms, such as Telegram, will enable prompt notifications upon threat detection, empowering users to take immediate action to safeguard their IoT networks. Overall, the proposed system represents a comprehensive and efficient solution to bolster the cybersecurity posture of IoT environments.

3.4 MODULE IDENTIFICATION

- Raspberry Pi Integration Module
- Attack Simulation Module
- Telegram Integration Module

3.4.1 Raspberry Pi Integration Module:

Integrate Snort IDS seamlessly with the Raspberry Pi, adapting configurations for optimal performance in resource-constrained IoT environments.

3.4.2 Attack Simulation Module:

Simulate various cyber threats, including ICMP, SYN, and HTTP floods, brute force attempts, Nmap scans, and ARP spoofing, to assess Snort's detection capabilities.

3.4.3 Telegram Integration Module:

Integrate Telegram API with the Raspberry Pi to enable real-time alerting and notification functionality

3.5 SYSTEM REQUIREMENT

- **Raspberry Pi (RPi) Hardware:** The system requires one or more Raspberry Pi devices to serve as the host platform for deploying the Snort IDS. The specific RPi model chosen should meet the minimum hardware specifications necessary to support Snort and handle network traffic effectively.
- **Network Connectivity:** Stable network connectivity is essential for the Raspberry Pi devices to monitor network traffic and communicate with external systems, such as receiving alerts or updates. Wired Ethernet or reliable Wi-Fi connections can be used based on the deployment environment.

- **Operating System:** The Raspberry Pi devices should be equipped with a compatible operating system (OS) capable of running Snort IDS. Common choices include Raspbian (based on Debian), Ubuntu, or other Linux distributions optimized for IoT devices.
- **Snort IDS Software:** The Snort IDS software package needs to be installed and configured on each Raspberry Pi device. This includes setting up the necessary rules, configurations, and network interfaces to monitor and analyze incoming network traffic for potential threats.
- **Storage:** Sufficient storage space is required on the Raspberry Pi devices to store Snort logs, configuration files, and any additional data generated during the monitoring process. External storage options such as microSD cards or USB drives may be utilized to expand storage capacity if needed.
- **Power Supply:** Reliable power sources, such as USB power adapters or power over Ethernet (PoE) solutions, are necessary to ensure the uninterrupted operation of the Raspberry Pi devices, especially in deployments where continuous monitoring is essential.
- **Optional Components:** Depending on specific deployment requirements, additional components such as sensors, cameras, or supplementary security software may be integrated with the Raspberry Pi-based IDS solution to enhance functionality or address specific security needs.

3.6 DESIGN PROCESS AND EXPLANATION

The design process for implementing the proposed cybersecurity solution leveraging Snort IDS on the Raspberry Pi platform involves several key steps aimed at ensuring the effectiveness, efficiency, and reliability of the solution. Below is an overview of the design process along with explanations of each step:

- **Requirements Gathering:** The first step in the design process is to gather and analyze the requirements of the cybersecurity solution. This involves understanding the specific security challenges faced by IoT environments, identifying the desired functionalities and capabilities of the Snort IDS deployment, and determining the constraints and limitations of the Raspberry Pi platform.
- **System Architecture Design:** Based on the requirements gathered, the next step is to design the overall system architecture. This includes defining the solution's

components, such as the Raspberry Pi devices, network infrastructure, Snort IDS software, and any additional hardware or software components. The system architecture should outline how these components interact and communicate to achieve the desired security objectives.

- **Hardware Selection and Configuration:** With the system architecture defined, the appropriate hardware components, particularly the Raspberry Pi devices, need to be selected and configured. This involves choosing the RPi model(s) that meet the performance and resource requirements of running Snort IDS, setting up network connectivity, and ensuring adequate power supply for continuous operation.
- **Software Installation and Configuration:** The Snort IDS software, along with any necessary dependencies and supporting tools, must be installed and configured on the Raspberry Pi devices. This includes configuring network interfaces, defining detection rules and policies, specifying logging and alerting mechanisms, and fine-tuning performance parameters to optimize IDS operation.
- **Integration with Alerting Mechanisms:** To enable timely detection and response to security threats, the Snort IDS deployment needs to be integrated with alerting mechanisms. This typically involves configuring the IDS to generate alerts or notifications when suspicious activity is detected and setting up communication channels for delivering alerts to administrators or security personnel through telegram BOT.
- **Testing and Validation:** Once the Snort IDS deployment is configured, thorough testing and validation are essential to ensure its effectiveness in detecting and mitigating cyber threats. This includes conducting simulated attacks, analyzing IDS alerts and responses, evaluating performance under different traffic conditions, and verifying that the solution meets the specified security requirements.
- **Documentation and Training:** Finally, comprehensive documentation should be created to document the design, configuration, and operation of the Snort IDS deployment on the Raspberry Pi platform. This documentation serves as a reference for administrators and operators responsible for maintaining the system.

CHAPTER 4

SYSTEM METHODOLOGIES

4.1 MODULE EXPLANATIONS

4.1.1 Raspberry Pi Integration Module

a) Installing an Operating System on Raspberry Pi

To begin using your Raspberry Pi, you'll need to install an operating system. Typically, this involves writing an operating system image onto a storage device such as a microSD card. Here's how to do it:

- Obtain a computer with an SD card reader and imaging software.
- Insert the microSD card into the computer.
- Use Raspberry Pi Imager, a versatile tool available for macOS, Windows, and Linux, to download and write operating system images onto the microSD card. Raspberry Pi Imager supports a variety of operating systems and allows for the configuration of credentials and remote access settings.
- Alternatively, you can manually download operating system images and write them onto the microSD card using imaging software.

b) Snort Configuration

To install Snort on Raspberry Pi, execute the following command in the terminal:

sudo apt-get install snort

Once installed, Snort relies on configuration files to define its behavior. The main configuration file, typically named `snort.conf`, is located in the `/etc/snort` directory. This file contains global settings such as logging options, preprocessors, and rule locations. You can modify this file using a text editor like Nano or vi:

sudo nano /etc/snort/snort.conf

Detection rules, also known as rule-based algorithms, are crucial for identifying suspicious or malicious network traffic. These rules are written in a specific format and are typically stored in separate rule files within the `/etc/snort/rules` directory. You can create custom rules or use pre-existing rule sets provided by Snort or third-party sources. To manage rule files, use a text editor:

sudo nano /etc/snort/rules/local.rules

By modifying these configuration files and rules, you can tailor Snort to your specific network security needs while ensuring effective threat detection and response.

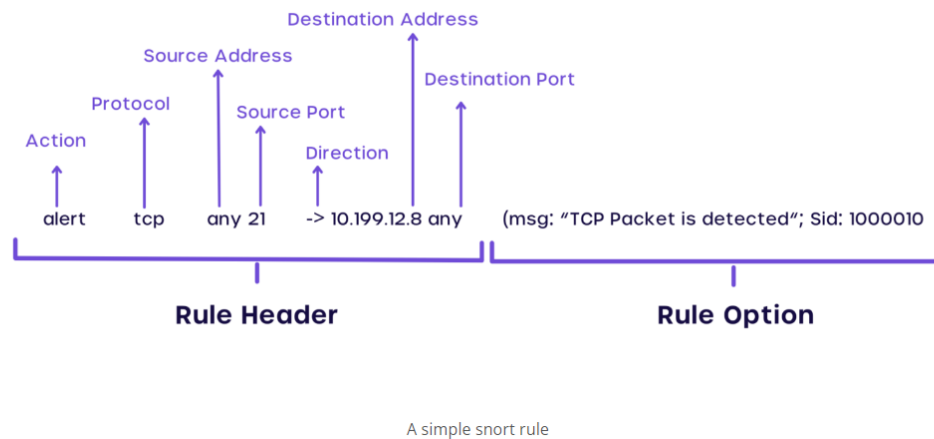


Figure 4.1: Snort Rule Explanation

4.1.2. Attack Simulation Module

The attack simulation module is a crucial component in the testing framework designed to evaluate the efficacy of the Snort IDS in detecting and mitigating various cyber threats. This module comprises a collection of tools, scripts, and methodologies aimed at simulating real-world attack scenarios within a controlled environment.

At its core, the attack simulation module leverages a combination of existing tools and custom scripts to generate simulated attack traffic representative of common cyber threats encountered in IoT environments. These tools allow us to emulate a diverse range of attack vectors, including but not limited to ICMP floods, SYN floods, HTTP floods, brute force attempts, Nmap scans, and ARP spoofing attacks.

One key aspect of the attack simulation module is the customization and fine-tuning of attack parameters to closely mimic real-world scenarios. By adjusting parameters such as packet size, frequency, and payload content, we can replicate the nuanced behaviors exhibited by actual malicious actors, thereby creating a realistic testing environment for the Snort IDS.

Additionally, the attack simulation module facilitates the automation of attack scenarios through scripted workflows. This automation streamlines the testing process, enabling the rapid

execution of multiple attack scenarios and the systematic evaluation of the Snort IDS's detection and response capabilities across different threat vectors.

Furthermore, the attack simulation module provides comprehensive logging and reporting functionalities to capture detailed information about simulated attacks, including timestamps, packet payloads, and IDS alerts. This data allows for thorough analysis and validation of the IDS's performance under varying conditions, helping to identify strengths, weaknesses, and areas for improvement.

Some examples are:

- **Hping3:** Hping3 is a command-line tool used for network testing and analysis. It allows users to generate various types of TCP/IP packets and send them over a network, making it a versatile tool for network exploration, troubleshooting, and security testing. Hping3 supports a wide range of functionalities, including Packet crafting: Hping3 enables users to create customized packets with specific characteristics such as source and destination IP addresses, ports, packet flags, and payloads. Network scanning: Hping3 can be used to perform different types of network scans, including SYN scans, UDP scans, and ICMP scans. These scans help users discover hosts, open ports, and network services on target systems. Security testing: Hping3 is commonly used for security assessments and penetration testing. It can simulate various types of network attacks, such as SYN floods, UDP floods, and ICMP floods, to evaluate the resilience of network defenses against potential threats. Tracerouting: Hping3 includes traceroute functionality, allowing users to trace the route packets take from their source to a destination host. This feature helps diagnose network connectivity issues and identify routing paths across the internet. Command Example: `hping3 -c 1000 -S -p 80 <target_IP>`
- **Nmap:** Nmap, also known as "Network Mapper," is a highly versatile and widely used open-source network scanning tool. Its primary function is to explore and map networks, providing valuable insights into network topology, device discovery, and service identification. One of Nmap's key features is its ability to perform comprehensive host discovery, allowing users to identify live hosts on a network through various techniques such as ICMP ping, TCP SYN scans, and ARP requests. Once live hosts are identified, Nmap can conduct port scans to determine which ports are open and what services are running on those ports. Port scanning with Nmap is

highly customizable, supporting different scan types including TCP connect scans, SYN scans, UDP scans, and FIN scans. Users can specify specific port ranges or scan all 65,535 ports to gather detailed information about network services and potential vulnerabilities. In addition to its scanning capabilities, Nmap offers powerful scripting capabilities through its Nmap Scripting Engine (NSE). This scripting engine allows users to write and execute custom scripts to automate tasks, gather additional information, and perform advanced network scans. Command Example: `nmap -sS -p 1-1000 <target_IP>`

- **Ettercap:** Ettercap is a comprehensive suite for man-in-the-middle attacks on LAN. It can intercept, sniff, and manipulate network traffic in real time. Ettercap provides features for ARP spoofing, packet sniffing, and protocol analysis, making it useful for simulating ARP spoofing attacks. Command Example: `sudo ettercap -T -q -i en1 -w dump -M ARP /192.168.0.4/ /192.168.0.1/`
- **Hydra:** Hydra is a powerful and versatile password-cracking tool that is commonly used in penetration testing and security assessments. It is designed to perform brute-force attacks against various network protocols, attempting to guess passwords by systematically trying different combinations until the correct one is found. Hydra supports a wide range of protocols and services, including FTP, HTTP, SSH, Telnet, SMB, SNMP, LDAP, MySQL, PostgreSQL, and more. The tool is highly configurable, allowing users to specify parameters such as target IP addresses, port numbers, usernames, password lists, and attack types. Hydra can be used to assess the strength of password-based authentication mechanisms and identify weak or easily guessable passwords. However, it should only be used in ethical hacking scenarios with proper authorization, as unauthorized use can be illegal and unethical. Command Example: `hydra -l admin -P passwords.txt ftp://<target_IP>`

4.1.3. Telegram Integration Module

To integrate Telegram alerting functionality into the Raspberry Pi (RPI) environment, the first step involves creating a Telegram bot using the BotFather service provided by Telegram. BotFather guides users through the bot creation process, generating a unique API token that serves as the authentication mechanism for the bot. This token is essential for establishing communication between the bot and the Telegram platform.

Once the bot is created, the next step is to obtain the chat ID of the desired Telegram channel or user where alert notifications will be sent. This chat ID serves as the destination for the alert messages generated by the bot. Users can retrieve their chat ID by sending a message to the bot and inspecting the response, or by utilizing third-party tools designed for this purpose.

With the API token and chat ID in hand, the next phase involves integrating these keys into a Python script that will facilitate the alerting process. The provided Python script demonstrates how to send alert messages to Telegram using the requests library to make HTTP POST requests to the Telegram Bot API endpoint. The script continuously monitors a designated log file, which contains Snort IDS alert logs or other security event data.

In the monitoring loop of the Python script, the log file is checked for updates, and if new alerts are detected, customized alert messages are composed based on predefined patterns or signatures. These alert messages include details such as the timestamp of the event and a descriptive message indicating the type of attack detected. The script then sends these alert messages to the specified Telegram chat ID, ensuring that administrators receive timely notifications of security incidents.

By running the Python script on the Raspberry Pi, organizations can leverage Telegram as a convenient and efficient communication channel for receiving real-time alert notifications. This integration enhances incident response capabilities, enabling administrators to promptly identify and mitigate security threats in IoT environments.

CHAPTER 5

IMPLEMENTATION

5.1 Raspberry Pi Integration Module

5.1.1 Installing an operating system on RPi

To set up Raspberry Pi OS, we can use the Raspberry Pi Imager tool. It's a straightforward process where we can download and install Raspberry Pi Imager on a computer with an SD card reader. After inserting the SD card into the reader, launch Raspberry Pi Imager and follow the following steps.

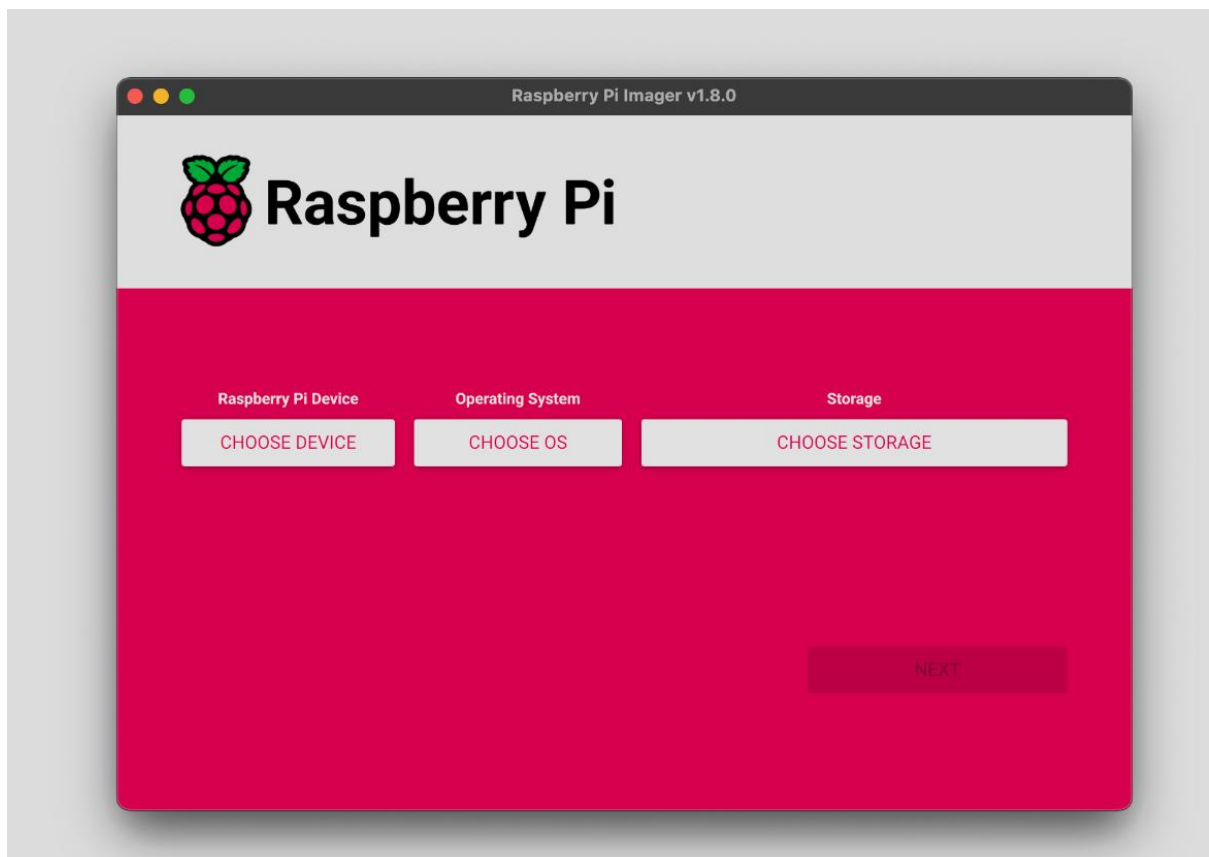


Figure 5.1: Raspberry Pi Imager

Click **Choose Device** and select the Raspberry Pi 4 model from the list.

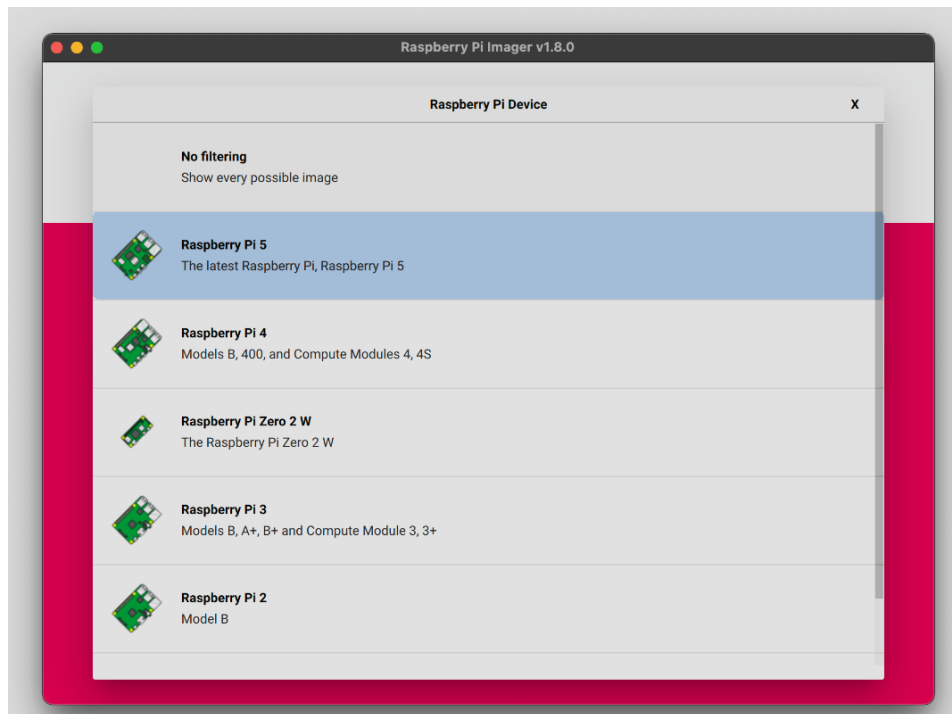


Figure 5.2: Choosing a Device

Next, click **Choose OS** and select Raspberry Pi OS (64-bit) to install.

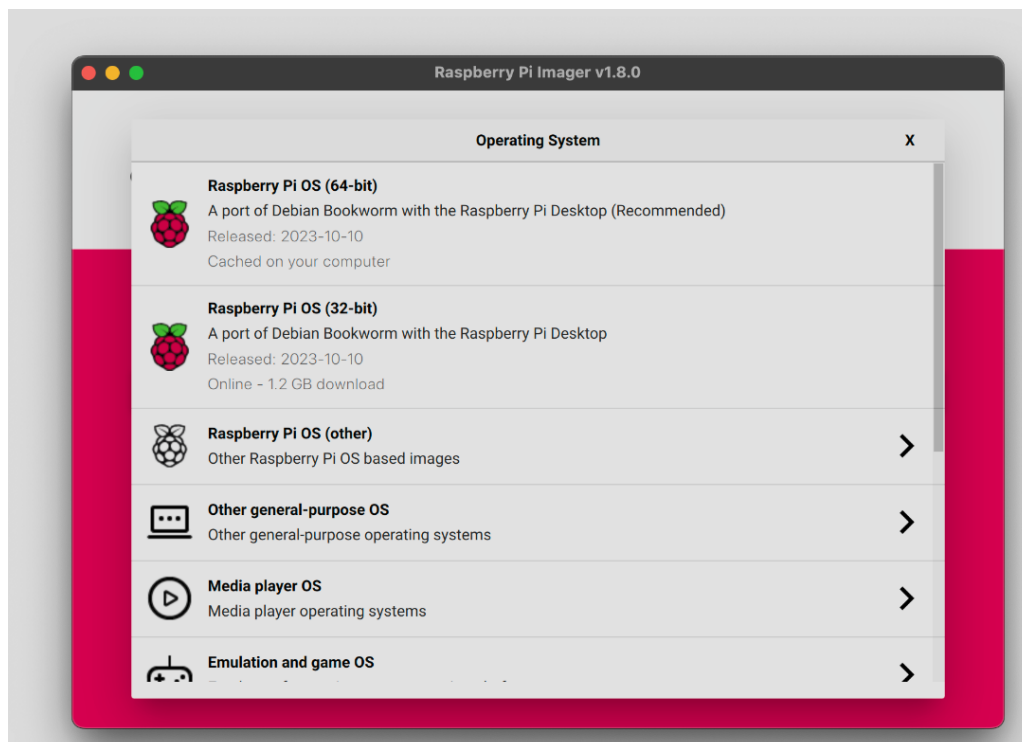


Figure 5.3: Choosing an OS

Select the storage device

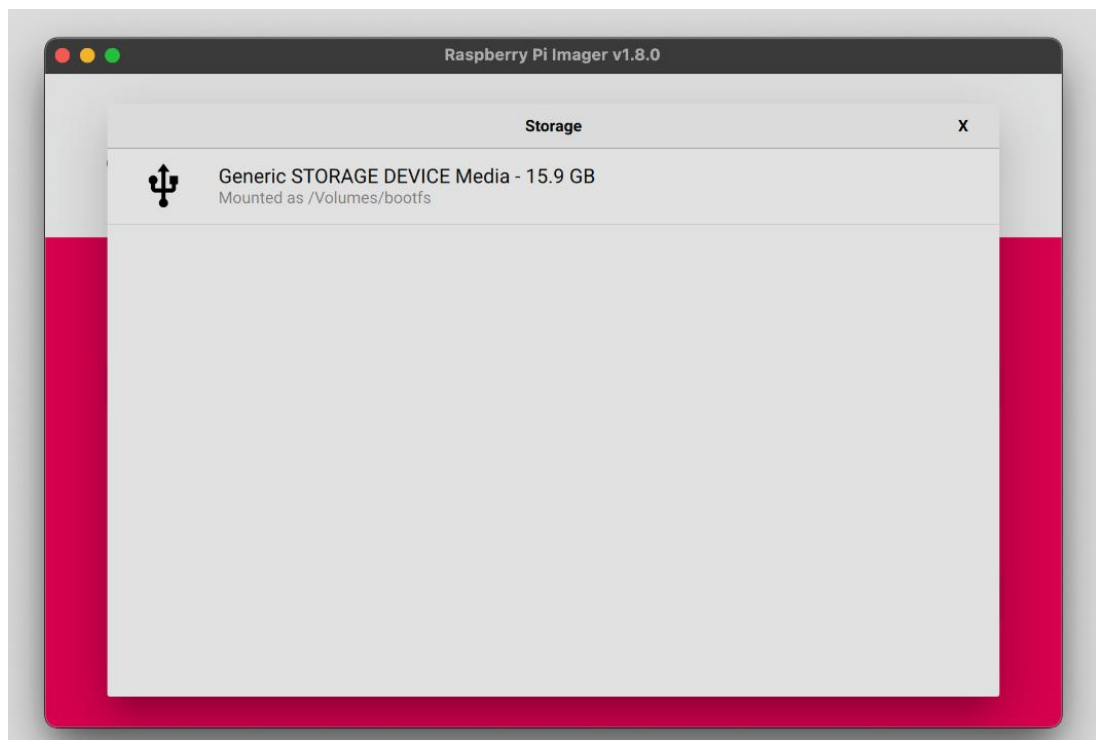


Figure 5.4: Storage Devices

*Next, click **Next**.*

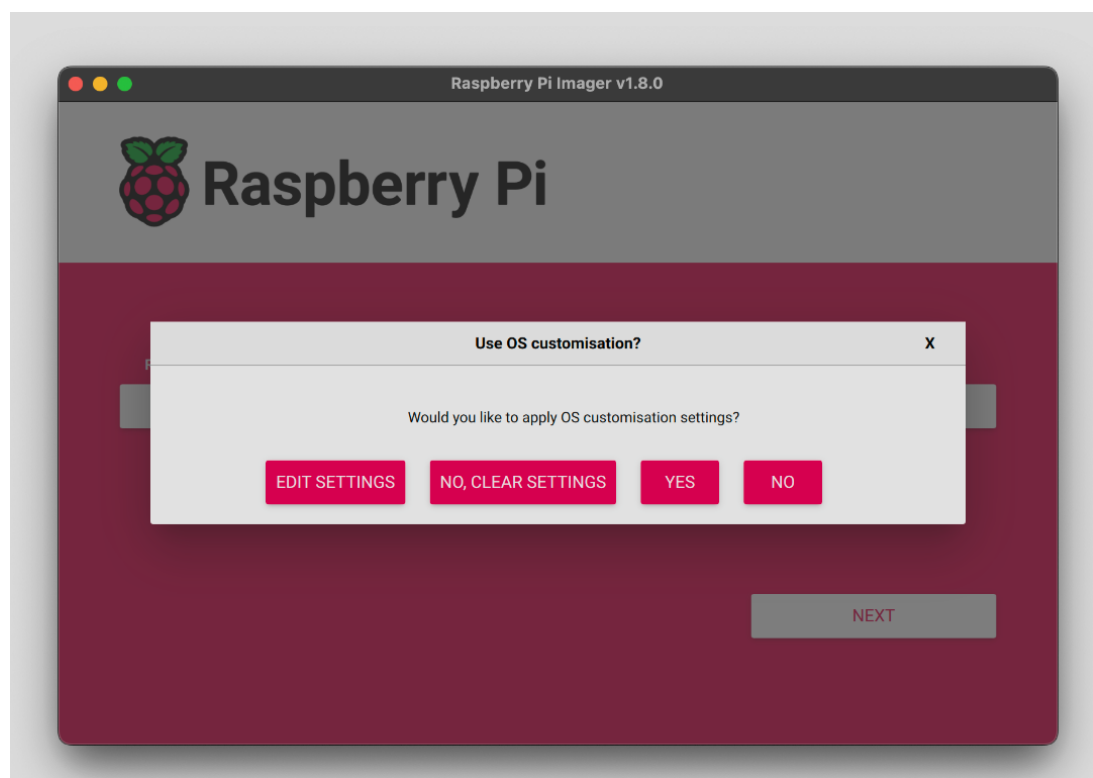


Figure 5.5: OS Customization Option

Before you can write data to the storage device, you will need to respond "Yes" to the "Are you sure you want to continue?" popup. If you encounter an admin prompt requesting

permission to read and write to your storage medium, please grant Imager the necessary permission to proceed.



Figure 5.6: Writing an Image

Once the "Write Successful" popup appears, the image is ready to boot on a Raspberry Pi.

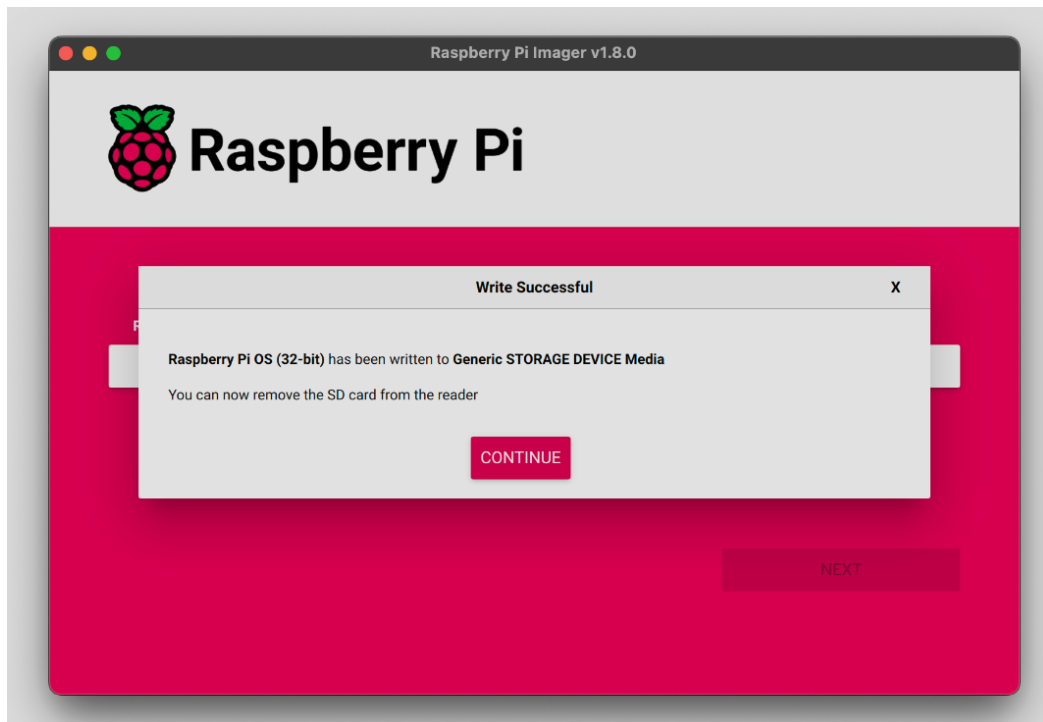


Figure 5.7 Ready to boot

5.1.2 Snort configuration:

To configure Snort on the Raspberry Pi, we first need to install the Snort package. This can be done using the following command:

sudo apt-get install snort

```

kali@raspberrypi: ~ $ sudo apt-get install snort
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
snort is already the newest version (2.9.20-0+deb11u1).
The following packages were automatically installed and are no
libopengl0 libre2-9 sse3-support
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 32 not upgraded.

```

Figure 5.8: Snort Installation

Once Snort is installed, we need to configure its main configuration file, typically located at `/etc/snort/snort.conf`. This file contains global settings, logging options, preprocessors, and rule locations. We can use a text editor like Nano or Vi to modify this file:

sudo nano /etc/snort/snort.conf

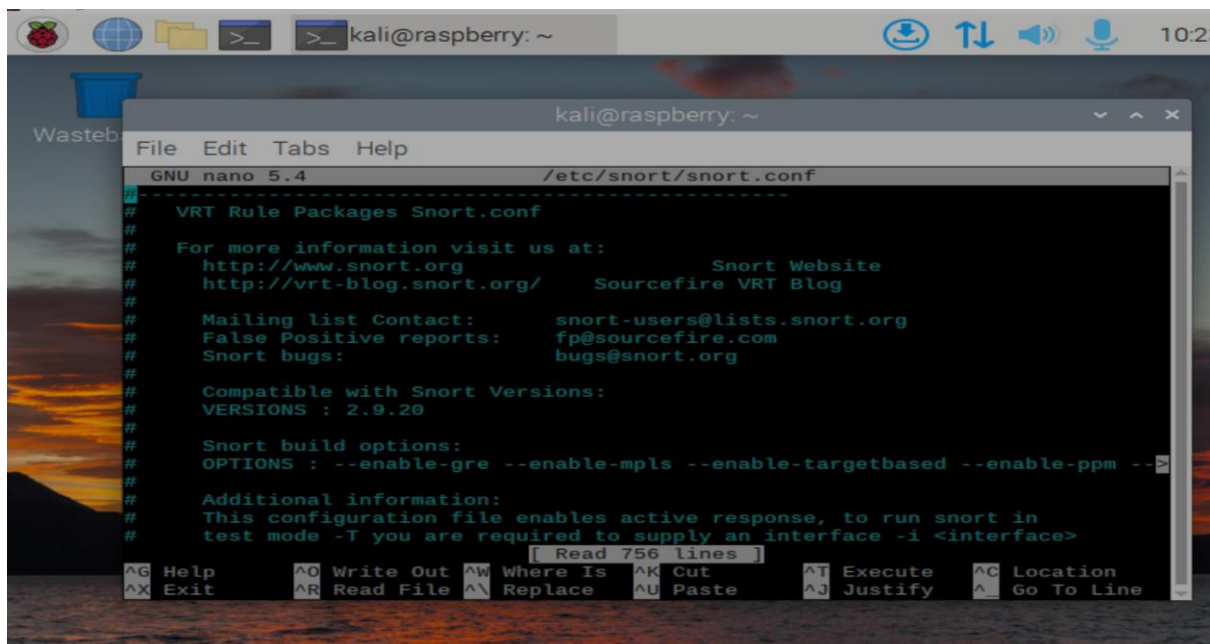


Figure 5.9: Snort Configuration

Within the configuration file, we can specify parameters such as network interfaces to monitor, logging settings, and rule files to include. Additionally, we can enable or disable preprocessors and set alerting thresholds.

For defining detection rules, which are crucial for identifying suspicious or malicious network traffic, we typically use separate rule files stored in the `/etc/snort/rules` directory. The main rule file is often named `local.rules`. We can create custom rules or use pre-existing rule sets provided by Snort or third-party sources. To create or modify rule files, we can again use a text editor:

sudo nano /etc/snort/rules/local.rules

Once the configuration is complete, we can start the Snort service and begin monitoring network traffic for potential threats.

5.1.3 Raspberry Pi Placement

Strategic placement of the Raspberry Pi (RPI) within the network topology is crucial for effective intrusion detection and monitoring. Ideally, the RPI should be connected to a switch in the home network, enabling it to capture and analyze all network traffic packets traversing

the network. This placement ensures comprehensive visibility into network activity, allowing the IDS to detect and respond to potential intrusions in real-time. The utilization of a Switch Port Analyzer (SPAN) port in the switch facilitates the mirroring of network traffic to the RPI, providing it with the necessary data for accurate and timely threat detection. By positioning the RPI at a vantage point within the network infrastructure, organizations can enhance their security posture and mitigate risks associated with unauthorized access and malicious activities

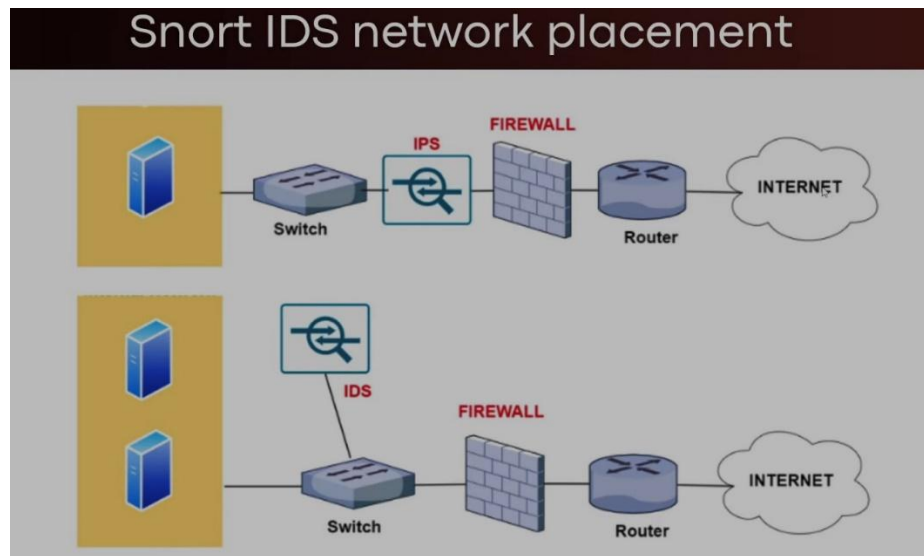


Figure 5.10: RPI Placement

5.2 Attack Simulation Module

The Attack Simulation Module is implemented using Kali Linux as the operating system, chosen for its extensive suite of pre-installed penetration testing tools and robust support for network attack simulations. The following tools are utilized for conducting various attack simulations:

- **Hping3:** Hping3 is a command-line tool available in Kali Linux for generating TCP/IP packets and performing network testing. It supports various types of scans and attacks, including SYN floods, UDP floods, and ICMP floods.

To install Hping3 on Kali Linux, the following command is used:

```
sudo apt-get install hping3
```

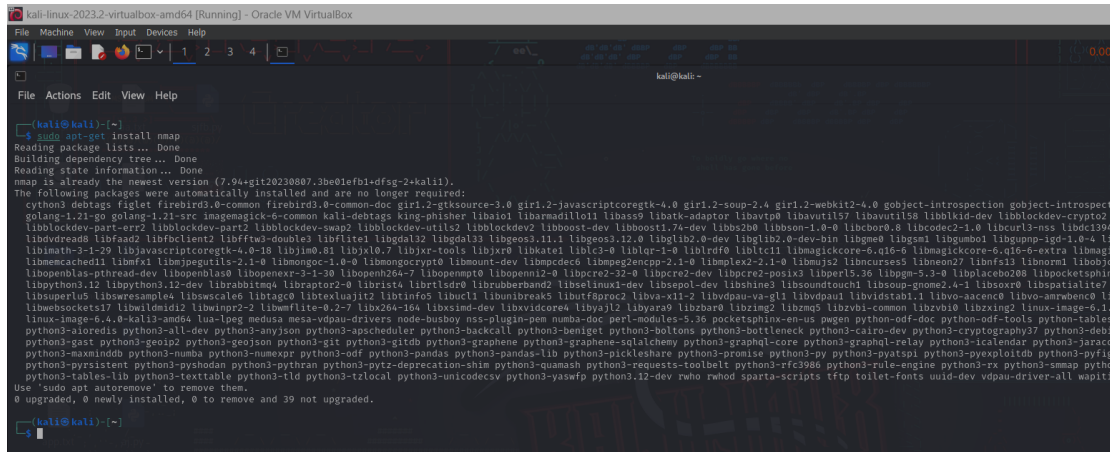



Figure 5.13: Nmap installation

An example command for performing a SYN scan using Nmap is as follows:

nmap -sS -p 1-1000 <target_IP>

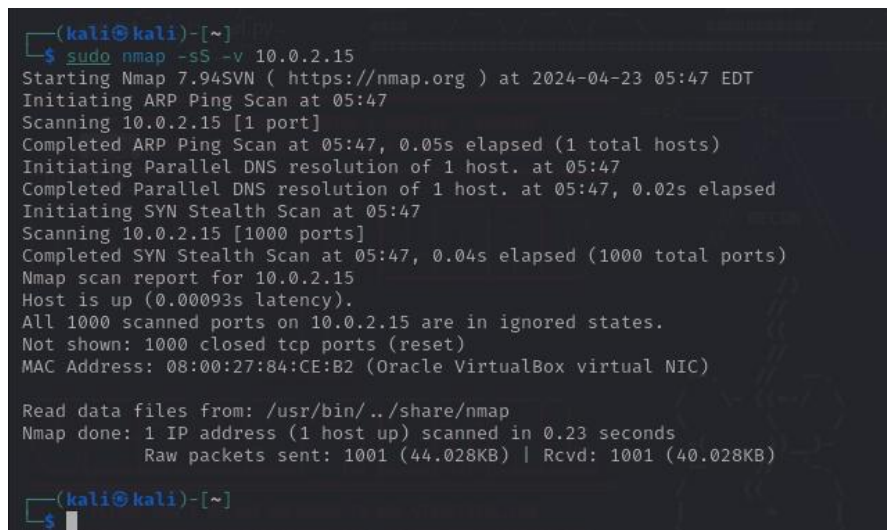


Figure 5.14: Nmap Attack

- **Etercap:** Ettercap is a comprehensive suite available in Kali Linux for man-in-the-middle attacks on LAN. It can intercept, sniff, and manipulate network traffic in real time, making it useful for simulating ARP spoofing attacks.

To install Ettercap on Kali Linux, the following command is used:

sudo apt-get install ettercap-text-only

```

(kali@kali)-[~]
$ sudo apt install ettercap-text-only
Reading package lists ... Done
Building dependency tree ... Done
Reading state information ... Done
The following packages were automatically installed and are no longer req
cython3 debtags figlet gir1.2-gtksource-3.0 gir1.2-javascriptcoregtk-4.0
gobject-introspection-bin golang-1.21 golang-1.21-doc golang-1.21-go go
libarmadillo11 libass9 libatk-adaptor libavtp0 libavutil57 libblkid-dev
libblockdev-part2 libblockdev-swap2 libblockdev-utils2 libblockdev2 lib
libdavid6 libdc1394-25 libdca0 libdirectfb-1.7-7 libdvdnav4 libdvdread8
libgeos3.12.0 libglb2.0-dev libglb2.0-dev-bin libgme0 libgumbo1 libgu
libjim0.81 libjxr-tools libjxr0 libkate1 liblc3-0 liblqr-1-0 liblrd0 l
libmbdecrypto7 libmfx1 libmjpegutils-2.1-0 libmount-dev libmpcdec6 libm
libobjc-12-dev libopenblas-dev libopenblas-pthread-dev libopenblas0 lib
libpcre2-posix3 libperl5.36 libpgm-5.3-0 libplacebo208 libpocketsphinx3
libraptor2-0 librist4 librtlsdr0 librubberband2 libselinux1-dev libsepo
libstr1.5-gnutls libstrtp2-1 libsuperlu5 libswscale6 libtagc0 libtexluaaj
libvo-amrwbenc0 libvpx7 libwebkit2gtk-4.0-37 libwebsockets17 libwildmid
libzxing2 linux-image-6.1.0-kali9-amd64 linux-image-6.4.0-kali3-amd64 l
pocketsphinx-on-us-pwgen-python-odf-doc-python-odf-tools-python-tables

```

Figure 5.15: Ettercap installation

An example command for performing an ARP spoofing attack using Ettercap is as follows:

sudo ettercap -T -q -i en1 -w dump -M ARP /192.168.0.4/ /192.168.0.1/

```

Tosch:ettercap Tosch$ sudo ettercap -Tq -i en1 -w dump-ettercap -M ARP /192.168.0.16/ /192.168.0.1/
Password:
Destination      Protocol Info
195.144.11.44    TCP      50147 > http [FIN, ACK] Seq=2466 Ack=6
ettercap NG-0.7.3 copyright 2001-2004 ALor & NaGA      ARP      192.168.0.1 is at 00:25:00:
Listening on en1... (Ethernet)                          ARP      192.168.0.16 is at 00:25:00:
195.144.11.44    TCP      50152 > http [SYN] Seq=0 Win=8192 Len=
en1 -> 00:25:00: 192.168.0.7 255.255.255.0
195.144.11.44    TCP      50152 > http [SYN] Seq=0 Win=8192 Len=
SSL dissection needs a valid 'redir_command_on' script in the etter.conf file> http [ACK] Seq=1 Ack=1 Win=175
Privileges dropped to UID      GID      ...      HTTP      POST /users/login HTTP/1.1 (applicati
0 plugins (disabled by configure...)      TCP      50152 > http [ACK] Seq=1 Ack=1 Win=175
39 protocol dissectors      144.11.44      HTTP      [TCP Retransmission] POST /users/login
53 ports monitored      44.11.44      HTTP      GET /posts HTTP/1.1
7587 mac vendor fingerprint      144.11.44      HTTP      [TCP Retransmission] GET /posts HTTP/1
1698 tcp OS fingerprint      144.11.44      HTTP      GET /users/login HTTP/1.1
2183 known services      195.144.11.44      HTTP      GET /users/login HTTP/1.1
Scanning for merged targets (2 hosts)...
* |=====| 100.00 %
4      PHPSESSID=0d6      ; __utma=195234862
1 hosts added to the hosts list...
form of encoded (r)
ARP poisoning victims:
GROUP 2 : 192.168.0.1 04:30:C7:
Starting Unified sniffing...

```

Figure 5.16: Ettercap attack

- **Hydra:** Hydra is a powerful password-cracking tool capable of performing brute-force attacks against various network protocols. It is used for simulating attacks such as FTP brute force. To install Hydra on Kali Linux, the following command is used:

sudo apt-get install hydra

```

(kali@kali)-[~/project1]
$ sudo apt-get install hydra
[sudo] password for kali:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
hydra is already the newest version (9.5-1).
hydra set to manually installed.
The following packages were automatically installed and are no longer needed:
  cython3 debtags kali-debtags libboost-dev libboost1.74-dev libboost1.74-stdlib
  libqt5multimedia5 libqt5multimedia5-plugins libqt5multimedia5-gst-plugins-base
  python3-backcall python3-beniget python3-debian python3-future python3-unicodcsv
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 148 not upgraded.

```

Figure 5.17: Hydra installation

An example command for simulating an FTP brute force attack using Hydra is as follows:

hydra -l admin -P passwords.txt ftp://<target_IP>

These tools, along with custom scripts, are employed to automate the execution of attack scenarios and simulate various cyber threats within a controlled environment. Comprehensive logging and reporting functionalities are integrated to capture detailed information about simulated attacks, facilitating thorough analysis and validation of the IDS's performance.

```

(kali@DESKTOP-SK08UEQ)-[/mnt/c/Users/RAJ/Desktop/javascript]
$ hydra -l msfadmin -P pass.txt 192.168.29.135 ssh -t 4
Hydra v9.2 (c) 2021 by van Hauser/THC & David Maciejak - Please do not
-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2022-07
[DATA] max 4 tasks per 1 server, overall 4 tasks, 4 login tries (l:1/p
[DATA] attacking ssh://192.168.29.135:22/
[22][ssh] host: 192.168.29.135 login: msfadmin password: msfadmin
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2022-07

```

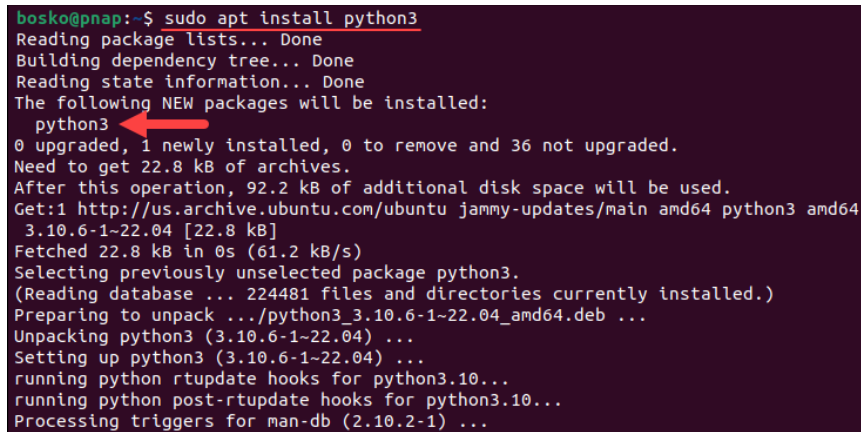
Figure 5.18: Hydra Attack

5.3 Telegram Integration Module

Integrate Telegram API with the Raspberry Pi to enable real-time alerting and notification functionality. Develop Python scripts to facilitate communication between Snort IDS and Telegram, allowing for instant notification of detected threats and security events. Implement custom alert formats and delivery mechanisms to ensure efficient dissemination of critical security information to designated administrators or stakeholders via Telegram messaging channels.

5.3.1 Installing python

Begin by installing Python 3 on your Raspberry Pi by executing the command "sudo apt install python3" in the terminal. Confirm the installation by entering "y" to proceed, after which the installation process will commence.



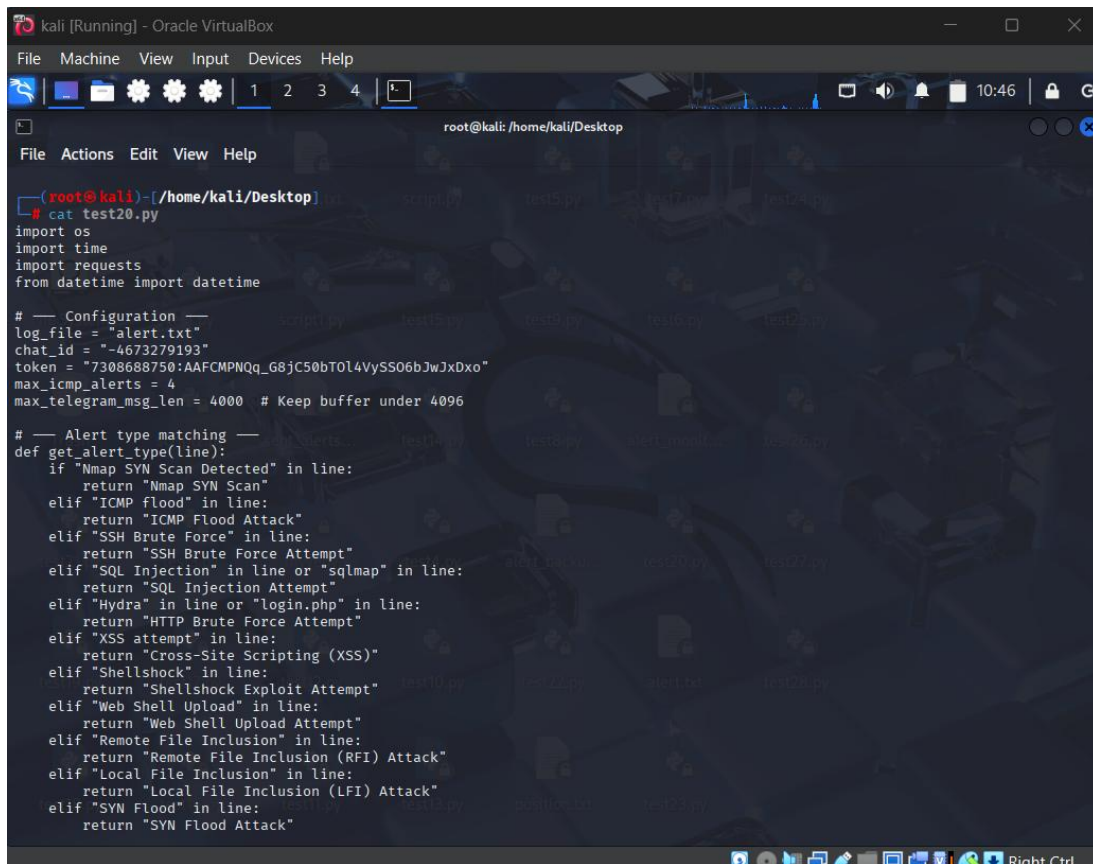
```
bosko@pnep:~$ sudo apt install python3
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  python3
0 upgraded, 1 newly installed, 0 to remove and 36 not upgraded.
Need to get 22.8 kB of archives.
After this operation, 92.2 kB of additional disk space will be used.
Get:1 http://us.archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3 amd64
  3.10.6-1~22.04 [22.8 kB]
Fetched 22.8 kB in 0s (61.2 kB/s)
Selecting previously unselected package python3.
(Reading database ... 224481 files and directories currently installed.)
Preparing to unpack .../python3_3.10.6-1~22.04_amd64.deb ...
Unpacking python3 (3.10.6-1~22.04) ...
Setting up python3 (3.10.6-1~22.04) ...
running python rtupdate hooks for python3.10...
running python post-rtupdate hooks for python3.10...
Processing triggers for man-db (2.10.2-1) ...
```

Figure 5.19: Python Installation

Once the installation completes, you can verify its success by checking the Python version using the command "python3 --version" in the terminal. This command will display the installed Python version, confirming that the installation was successful.

5.3.2 Python Script

- The Python script serves as the crucial link between the Snort IDS and Telegram, facilitating seamless communication for instant notification of detected threats and security events.
- Continuously monitoring a designated log file, the script swiftly responds to any updates, promptly detecting new alerts as they arise.
- Upon identification of a security incident, the script dynamically composes customized alert messages, leveraging predefined patterns or signatures to encapsulate essential information.
- These alert messages are swiftly dispatched to the specified Telegram chat ID, ensuring administrators receive timely notifications, empowering them to take swift action in response to security threats.



```
root@kali: /home/kali/Desktop
File Actions Edit View Help

(root@kali) - [~/home/kali/Desktop]
# cat test20.py
import os
import time
import requests
from datetime import datetime

# --- Configuration ---
log_file = "alert.txt"
chat_id = "-4673279193"
token = "7308688750:AAFCMPNqG8jC50bT0L4VySS06bJwJxDxo"
max_icmp_alerts = 4
max_telegram_msg_len = 4000 # Keep buffer under 4096

# --- Alert type matching ---
def get_alert_type(line):
    if "Nmap SYN Scan Detected" in line:
        return "Nmap SYN Scan"
    elif "ICMP flood" in line:
        return "ICMP Flood Attack"
    elif "SSH Brute Force" in line:
        return "SSH Brute Force Attempt"
    elif "SQL Injection" in line or "sqlmap" in line:
        return "SQL Injection Attempt"
    elif "Hydra" in line or "login.php" in line:
        return "HTTP Brute Force Attempt"
    elif "XSS attempt" in line:
        return "Cross-Site Scripting (XSS)"
    elif "Shellshock" in line:
        return "Shellshock Exploit Attempt"
    elif "Web Shell Upload" in line:
        return "Web Shell Upload Attempt"
    elif "Remote File Inclusion" in line:
        return "Remote File Inclusion (RFI) Attack"
    elif "Local File Inclusion" in line:
        return "Local File Inclusion (LFI) Attack"
    elif "SYN Flood" in line:
        return "SYN Flood Attack"
```

Figure 5.20: Python script

This script facilitates real-time communication between Snort IDS and Telegram, enabling instant notification of detected threats and security events. It continuously monitors a designated log file for updates, and upon detecting new alerts, composes customized alert messages based on predefined patterns or signatures. These alert messages are then sent to the specified Telegram chat ID, ensuring that administrators receive prompt notifications of security incidents.

5.3.3 Running a Script

- Open your terminal or command prompt window on your Raspberry Pi or desired system.
- Navigate to the directory where the "test.py" file is located. You can use the `cd` command followed by the directory path to move to the appropriate location.
- Once in the correct directory, type the command `python3 telegramalertscript.py` and press Enter. This command initiates the execution of the Python script, launching the monitoring process.

- The script will begin actively monitoring the designated log file, waiting vigilantly for any updates or new entries indicative of potential security threats.

```
roshiq@raspberrypi:~/telegram $ python3 telegramAlertscript.py
```

Figure 5.21: Running a script

5.3.4 Receiving an Alert

- As the script continuously monitors the log file, it remains on standby, ready to spring into action upon detecting suspicious activity.
- When a relevant security event occurs, such as a detected attack or anomaly, the script instantly formulates a detailed alert message containing pertinent information about the incident.
- This alert message is then dispatched directly to your Telegram account, specifically to the chat identified by the provided chat ID.
- Upon receiving the alert, you'll be promptly notified on your Telegram app or desktop client, enabling you to take immediate action to address the detected security issue.
- The alert message is crafted to provide essential details, including the type of attack, timestamp of the incident, and any additional context extracted from the log file.

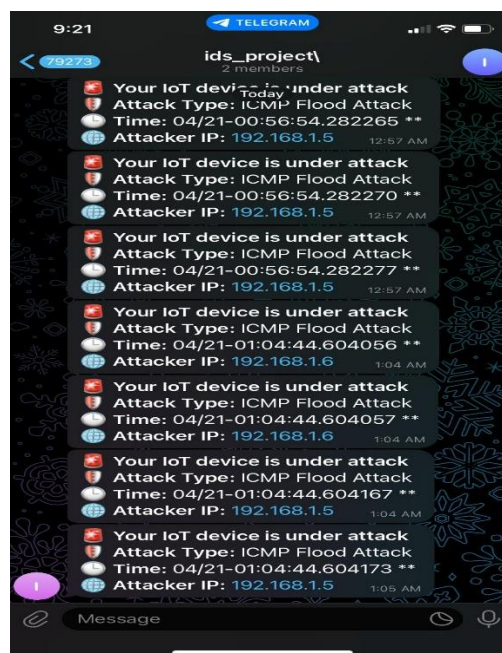


Figure 5.22: Alert Notification in Telegram

5.4 Performance Evaluation Module

In the Performance Evaluation Module, alerts generated by the Snort IDS are thoroughly examined to assess the system's effectiveness in detecting and responding to potential cyber threats. This evaluation includes the analysis of both false positives and false negatives to gauge the IDS's accuracy and reliability.

A sample alert generated by the Snort IDS during performance evaluation is as follows:

[] [1:1000001:1] SYN Flood Detected [**]**

[Classification: Attempted Denial of Service] [Priority: 1]

04/23-12:00:00.000000 [] [Priority: 0] {TCP} 192.168.1.100:80 -> 192.168.1.1:80**

This alert indicates the detection of a SYN flood attack targeting the destination IP address 192.168.1.1 on port 80. The classification labels the event as an "Attempted Denial of Service," with a priority level of 1.

During the evaluation process, the following criteria are considered:

5.4.1. False Positives: Alerts that incorrectly identify benign network traffic as malicious activity. These false alarms can lead to unnecessary alerts and potentially disrupt normal network operations. The Performance Evaluation Module scrutinizes false positives to identify any misconfigurations or inaccuracies in the IDS's rule sets or detection mechanisms.

5.4.2. False Negatives: Instances where the IDS fails to detect actual cyber threats, allowing malicious activities to go unnoticed. False negatives pose a significant risk as they indicate gaps in the IDS's detection capabilities, leaving the network vulnerable to attacks. The Performance Evaluation Module meticulously examines false negatives to pinpoint weaknesses in the IDS's rule sets, configuration, or monitoring parameters.

By meticulously analyzing alerts and scrutinizing false positives and false negatives, the Performance Evaluation Module provides valuable insights into the Snort IDS's performance, helping to fine-tune its configuration and enhance its efficacy in safeguarding the network against cyber threats.

5.5. Performance Improvement Module

In the Performance Improvement Module, one strategy involves adjusting existing rules to enhance detection accuracy and minimize false positives. For example, consider the following modification to a rule designed to detect SQL injection attacks:

Original Rule:

```
alert tcp any any -> any any (msg:"SQL Injection Detected"; content:"SELECT";  
nocase; content:"FROM"; nocase; sid:10001;)
```

Modified Rule:

```
alert tcp any any -> any any (msg:"SQL Injection Detected"; content:"SELECT";  
nocase; content:"FROM"; nocase; content:"UNION"; nocase; sid:10001;)
```

In this example, the original rule triggers an alert whenever a TCP packet contains the words "SELECT" and "FROM," which are indicative of SQL queries. However, this rule may generate false positives in scenarios where legitimate SQL queries are executed. To improve specificity, the rule is modified to include the keyword "UNION," a common component of SQL injection attacks. By refining the rule criteria, the IDS can more accurately identify suspicious SQL injection attempts while reducing false positives.

Additionally, new rules can be added to address emerging threats and vulnerabilities. For instance, consider the following rule designed to detect attempts to exploit a recently discovered vulnerability in a web application:

New Rule:

```
alert tcp any any -> any 80 (msg: "Web Application Vulnerability Exploit";  
content:"/vulnerable_endpoint"; http_uri; sid:10002;)
```

This rule monitors HTTP traffic for requests targeting a vulnerable endpoint ("/vulnerable_endpoint"). Upon detecting such traffic, the IDS generates an alert, allowing administrators to investigate and remediate the potential security issue. By proactively adding rules to detect known vulnerabilities, the IDS can effectively safeguard against zero-day exploits and emerging threats, bolstering the overall security posture of the network environment.

```

GNU nano 7.2 local.rules *
# $Id: local.rules,v 1.11 2004/07/23 20:15:04 bmc Exp $
# -----
# LOCAL RULES
# -----
# This file intentionally does not come with signatures. Put your local
# additions here.
alert tcp any any -> $HOME_NET 21 (msg:"FTP connection attempt"; sid:10000011; rev:1;)
alert tcp any any -> $HOME_NET 22 (msg:"SSH connection attempt"; sid:10000011; rev:1;)
alert tcp any any -> $HOME_NET 22 (msg:"SSH failed login"; content:"Permission denied";sid:10000012; rev:1;)
alert tcp any any -> $HOME_NET 21 (msg:"FTP failed login"; content:"Login incorrect";detection_filter:track by_src, count 2, seconds 60; sid:10000012; rev:1;)
alert icmp any any -> $HOME_NET any (msg:"ICMP ping detected"; sid:10000013; rev:1; classtype:icmp-event;)
alert tcp any any -> $HOME_NET any (msg:"TCP Port Scanning"; detection_filter:track by_src, count 30, seconds 60; sid:10000016; rev:2;)
alert tcp any any -> any 22 (msg:"SSH login failed attempt - Permission denied"; flow:to_server,established; content:"Permission denied"; depth:39; sid:10000016; rev:2;)
alert icmp any any -> $HOME_NET any (msg:"ICMP detected"; sid:10000021; rev:1;)
alert tcp any any -> $HOME_NET 5000 (msg:"Multiple failed login attempts"; uricontent:"/login"; pcre:"/(username|password){6}/i"; threshold: type three; sid:10000021; rev:1;)
alert tcp any any -> $HOME_NET 5000 (msg:"Multiple Login Attempts"; content:"POST /login HTTP/1.1";pcre:"/(username|password){6}/i";threshold: type three; sid:10000021; rev:1;)
alert tcp any any -> $HOME_NET any (msg:"Get Request Flood attempt"; content:"GET"; detection_filter:track by_dst, count 50, seconds 1; sid:293;)
alert tcp any any -> $HOME_NET any (msg:"Brute force login attempt"; \
content:"POST /login HTTP/1.1"; \
pcre:"/username=[a-zA-Z0-9]{6}/password=[a-zA-Z0-9]{6}/i"; \
detection_filter:track by_src, count 3, seconds 60; \
sid:1000004;)
alert ( msg: "TAG.LOG_PVT"; sid: 1; gid: 2; rev: 1; metadata: rule-type preproc ; classtype: not-suspicious ; )
alert ( msg: "BO_TRAFFIC_DETECT"; sid: 1; gid: 105; rev: 1; metadata: rule-type preproc, policy balanced-ips drop, policy security-ips drop ; classtype: traffic-analysis ; )
alert ( msg: "BO_CLIENT_TRAFFIC_DETECT"; sid: 2; gid: 105; rev: 1; metadata: rule-type preproc, policy balanced-ips drop, policy security-ips drop ; classtype: traffic-analysis ; )
alert ( msg: "BO_SERVER_TRAFFIC_DETECT"; sid: 3; gid: 105; rev: 1; metadata: rule-type preproc, policy balanced-ips drop, policy security-ips drop ; classtype: traffic-analysis ; )
alert ( msg: "BO_SHORT_BUFFER_ATTACK"; sid: 4; gid: 105; rev: 1; metadata: rule-type preproc, policy balanced-ips drop, policy security-ips drop ; classtype: traffic-analysis ; )
alert ( msg: "RPC_FRAG_TRAFFIC"; sid: 1; gid: 106; rev: 1; metadata: rule-type preproc, service sunrpc ; classtype: protocol-command-decode ; )
alert ( msg: "RPC_MULTIPLE_RECORD"; sid: 2; gid: 106; rev: 1; metadata: rule-type preproc, service sunrpc ; classtype: protocol-command-decode ; )
alert ( msg: "RPC_LARGE_FRAGSIZE"; sid: 3; gid: 106; rev: 1; metadata: rule-type preproc, service sunrpc, policy security-ips alert ; classtype: bad-unknown ; )
alert ( msg: "RPC_INCOMPLETE_SEGMENT"; sid: 4; gid: 106; rev: 1; metadata: rule-type preproc, service sunrpc, policy security-ips alert ; classtype: bad-unknown ; )

```

Figure 5.23: Local rules

5.5.1 Rule Optimization Strategies:

In addition to modifying existing rules, the Performance Improvement Module explores advanced optimization strategies to enhance the efficiency and effectiveness of intrusion detection rules. Techniques such as rule consolidation, where multiple rules are combined into a single rule to reduce processing overhead, are implemented to streamline rule management and improve system performance.

Another optimization strategy involves fine-tuning rule thresholds and parameters to achieve the desired balance between detection sensitivity and false positive rates. By adjusting rule parameters such as detection thresholds, timeout values, and alert priorities, the IDS can better prioritize and respond to security events based on their severity and impact.

5.5.2 Community Rule Integration:

The Performance Improvement Module harnesses the power of community-driven rule repositories, such as Snort's official Snort Subscriber Rule Set (SSRS) and open-source rule repositories, to augment the IDS's detection capabilities and enhance performance. These community rules provide a wealth of curated signatures and detection mechanisms contributed by cybersecurity experts and organizations worldwide.

By integrating community rules into the IDS's rule set, the module expands the scope of threat coverage and enhances the detection of sophisticated and emerging cyber threats. Community rules often include signatures for zero-day vulnerabilities, newly discovered malware variants, and advanced attack techniques not covered by standard rule sets.

Additionally, the Performance Improvement Module incorporates automated rule management and update mechanisms to streamline the deployment and maintenance of community rules. Automated rule updates ensure that the IDS remains up-to-date with the latest threat intelligence and detection mechanisms, minimizing the risk of overlooking critical security updates and patches.

Furthermore, the module implements rule optimization techniques, such as rule pruning and prioritization, to maximize the efficiency and effectiveness of community rules. By selectively enabling and fine-tuning community rules based on the organization's specific security requirements and threat landscape, the IDS can optimize resource utilization and reduce false positive rates.

Through continuous monitoring and evaluation of community rule performance, the module iteratively refines rule sets and adjusts detection strategies to align with evolving security priorities and operational objectives. By leveraging the collective expertise and contributions of the cybersecurity community, the Performance Improvement Module strengthens the overall resilience and effectiveness of the IDS against a wide range of cyber threats and attack vectors.

```

memory scale: KB
total memory: 85.8652
pattern memory: 20.3926
match list memory: 30.2422
transition memory: 33.8555
fast pattern only: 1
ppid: MaxRss diff: 3072
ppid: patterns loaded: 300
-----
cap DAQ configured to passive.
commencing packet processing
+ [0] wlan0
4/25-11:01:38.993376 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2530 -> 192.168.1.11:80
4/25-11:01:38.993423 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2531 -> 192.168.1.11:80
4/25-11:01:38.993466 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2532 -> 192.168.1.11:80
4/25-11:01:38.996468 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2533 -> 192.168.1.11:80
4/25-11:01:38.996580 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2534 -> 192.168.1.11:80
4/25-11:01:38.996629 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2535 -> 192.168.1.11:80
4/25-11:01:38.996674 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2536 -> 192.168.1.11:80
4/25-11:01:38.996720 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2537 -> 192.168.1.11:80

```

Figure 5.24: Attack Detection

5.6 IOT Environment:

5.6.1 Camera Module Integration:

The Raspberry Pi camera module enhances the IoT security system by providing real-time surveillance. Hosted via a Python script, it captures live footage or triggered images (e.g., motion detection) and streams them to a web interface, enabling remote monitoring. This replaces physical door control with digital oversight, improving accessibility and security verification.



Figure 5.25: Camera Module integration

5.6.2 Website Creation:

The web application, built using Python Flask, offers a user-friendly interface for managing the security system. It includes authentication to ensure only authorized users can access the dashboard. Through the website, users can view live or captured footage from the camera module, trigger manual snapshots, and monitor the secured area in real time. This interface enhances remote accessibility, replacing physical door control with digital surveillance for improved security management.

```
roshiq@raspberrypi:~$ python3 cam2.py
[0:04:08.914262609] [2091] INFO Camera camera_manager.cpp:327 libcamera v0.4.0+
53-29156679
[0:04:08.988534090] [2098] WARN RPISdn sdn.cpp:40 Using legacy SDN tuning - please consider moving SDN inside rpi.denoise
[0:04:08.994003498] [2098] INFO RPI vc4.cpp:447 Registered camera /base/soc/i2c0mux/i2c@1/ov5647836 to Unicam device /dev/media1 and ISP device /dev/media2
[0:04:08.994162257] [2098] INFO RPI pipeline_base.cpp:1121 Using configuration file '/usr/share/libcamera/pipeline/rpi/vc4/rpi_apps.yaml'
[0:04:09.016830201] [2091] INFO Camera camera.cpp:1202 configuring streams: (0) 640x480-XBGR8888 (1) 640x480-SGBRG10_CSI2P
[0:04:09.017809516] [2098] INFO RPI Vc4.cpp:622 Sensor: /base/soc/i2c0mux/i2c@1/ov5647836 - Selected sensor format: 640x480-SGBRG10_1X10 - Selected unicam format: 640x480-pGAA
* Serving Flask app 'cam2'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.6:5000
Press CTRL+C to quit
192.168.1.4 - - [08/May/2025 23:35:26] "GET / HTTP/1.1" 200 -
192.168.1.4 - - [08/May/2025 23:35:27] "GET /favicon.ico HTTP/1.1" 404 -
192.168.1.4 - - [09/May/2025 00:14:50] "POST / HTTP/1.1" 302 -
192.168.1.4 - - [09/May/2025 00:14:50] "GET /video HTTP/1.1" 200 -
192.168.1.4 - - [09/May/2025 00:18:40] "GET /video HTTP/1.1" 200 -
192.168.1.4 - - [09/May/2025 00:18:41] "GET /video HTTP/1.1" 200 -
192.168.1.4 - - [09/May/2025 00:18:59] "GET / HTTP/1.1" 200 -
192.168.1.4 - - [09/May/2025 00:18:59] "GET /favicon.ico HTTP/1.1" 404 -
```

Figure 5.26: flask server

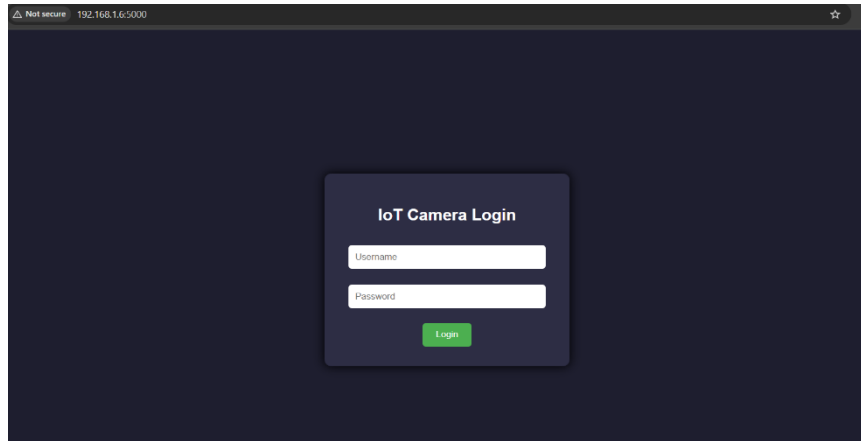


Figure 5.27: Authentication page

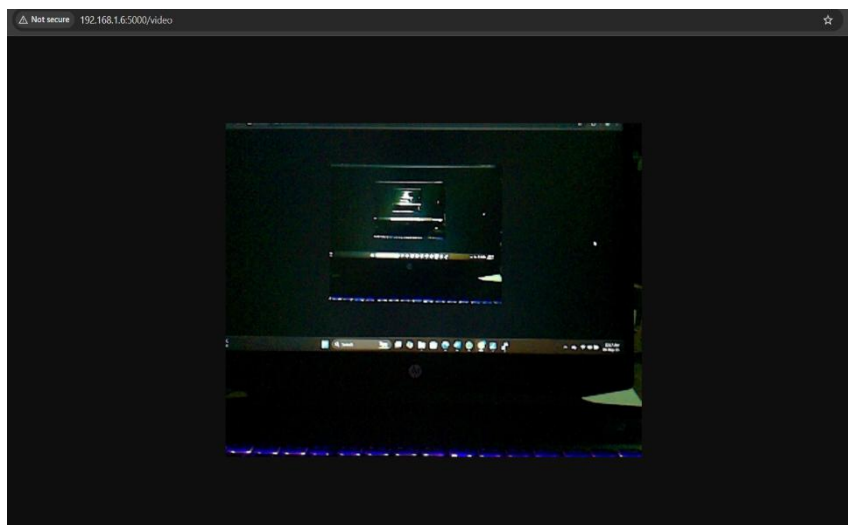


Figure 5.28: Streaming camera

5.8.3 Attacking and Alerting:

The testing phase involves simulating cyber attacks, such as brute force and denial-of-service (DoS) attacks, targeting the camera module's web interface. When Snort IDS detects an intrusion, it triggers an instant Telegram alert containing attack details (type, timestamp, and log data). This real-time notification enables administrators to respond swiftly, securing the surveillance system and mitigating potential breaches in the IoT environment.

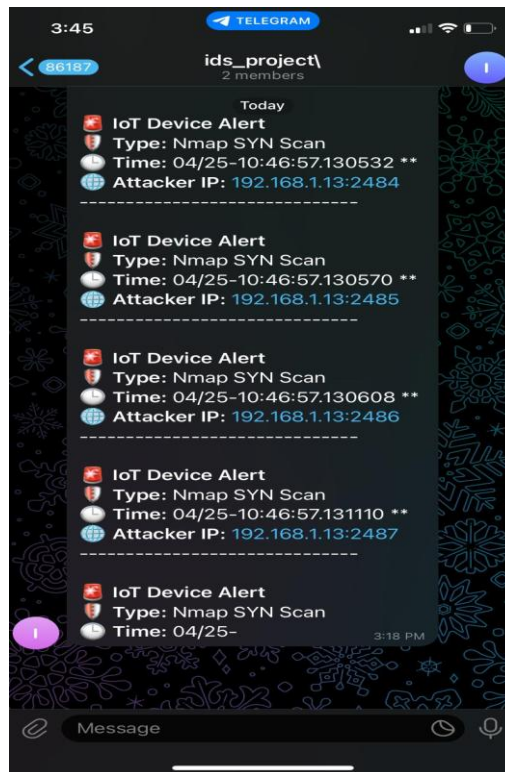


Figure 5.29: Alert

5.9 Code Overview:

5.9.1 Library Imports

```
from flask import Flask, Response, render_template_string, request, redirect, url_for
from picamera2 import Picamera2
import cv2
import os
import time
import requests
from datetime import datetime
```

- **Flask:** Web framework for hosting the camera interface and login system.
- **Picamera2:** Controls the Raspberry Pi camera module.
- **OpenCV (cv2):** Processes video frames for streaming.
- **Requests:** Sends HTTP alerts to Telegram.
- **Datetime:** Timestamps alerts and backups.

5.9.2 Telegram Alert

```
import os
import time
import requests
from datetime import datetime

# --- Configuration ---
log_file = "alert.txt"
chat_id = "-4673279193"
token = "7308688750:AAFCCMPNQq_G8jC50bTOl4VySSO6bJwJxDxo"
max_icmp_alerts = 4
max_telegram_msg_len = 4000 # Keep buffer under 4096

# --- Alert type matching ---
def get_alert_type(line):
    if "Nmap SYN Scan Detected" in line:
        return "Nmap SYN Scan"
    elif "ICMP flood" in line:
        return "ICMP Flood Attack"
    elif "SSH Brute Force" in line:
        return "SSH Brute Force Attempt"
    elif "SQL Injection" in line or "sqlmap" in line:
        return "SQL Injection Attempt"
    elif "Hydra" in line or "login.php" in line:
        return "HTTP Brute Force Attempt"
    elif "XSS attempt" in line:
        return "Cross-Site Scripting (XSS)"
    elif "Shellshock" in line:
        return "Shellshock Exploit Attempt"
    elif "Web Shell Upload" in line:
        return "Web Shell Upload Attempt"
    elif "Remote File Inclusion" in line:
        return "Remote File Inclusion (RFI) Attack"
    elif "Local File Inclusion" in line:
        return "Local File Inclusion (LFI) Attack"
    elif "SYN Flood" in line:
        return "Nmap SYN Attack"
    elif "XMAS Scan" in line:
        return "XMAS Port Scan"
    elif "FIN Scan" in line:
        return "FIN Port Scan"
    elif "DNS Query Flood" in line:
        return "DNS Query Flood"
```



```

elif "DNS Amplification" in line:
    return "DNS Amplification Attack"
elif "SMB Brute Force" in line:
    return "SMB Brute Force Attempt"
elif "Bruteforce Attempt" in line:
    return "HTTP Brute Force Attempt"
return None

# --- Telegram alert sender (with batching + rate limit handling) ---
def send_telegram_message(msg):
    url = f"https://api.telegram.org/bot{token}/sendMessage"
    payload = {"chat_id": chat_id, "text": msg, "parse_mode": "HTML"}

    while True:
        res = requests.post(url, data=payload)
        print    if res.status_code == 200:
("☑ Alert sent")
            break
        elif res.status_code == 429:
            retry = res.json().get("parameters", {}).get("retry_after", 5)
            print(f"⌚ Rate limited. Retrying in {retry}s")
            time.sleep(retry)
        else:
            pass
            break

def chunk_and_send(message):
    while message:
        part = message[:max_telegram_msg_len]
        send_telegram_message(part)
        message = message[max_telegram_msg_len:]
        time.sleep(1) # Delay between chunks

# --- Main monitor ---
print("👁 Monitoring alert.txt...")
file_position = 0
icmp_counter = 0

while True:
    if os.path.exists(log_file):
        with open(log_file, "r+") as f:
            f.seek(file_position)
            new_lines = f.readlines()
            file_position = f.tell()

```

```

all_alerts = []
sent_lines = []

for line in new_lines:
    if "[**]" not in line:
        continue

    alert_type = get_alert_type(line)
    if not alert_type:
        continue

    if alert_type in ["ICMP Flood Attack", "SYN Flood Attack", "DNS Amplification
Attack", "DNS Query Flood"]:
        if icmp_counter >= max_icmp_alerts:
            continue
        icmp_counter += 1

    timestamp = line.split(" ")[0].replace("[", "")
    src_ip = "Unknown"
    if "->" in line:
        src_ip = line.split("->")[0].split("}")[1].strip()

    alert_msg = (
        f"\n 🚨 <b>IoT Device Alert</b>\n"
        f"🛡️ <b>Type:</b> {alert_type}\n"
        f"🕒 <b>Time:</b> {timestamp}\n"
        f"🌐 <b>Attacker IP:</b> {src_ip}\n"
        f"{'-'*30}"
    )
    all_alerts.append(alert_msg)
    sent_lines.append(line)

# Send batched message
if all_alerts:
    final_message = "\n".join(all_alerts)
    chunk_and_send(final_message)

# Backup and cleanup
os.makedirs("alert_backups", exist_ok=True)
bkp_name =
datetime.now().strftime("alert_backups/alerts_%Y%m%d_%H%M%S.txt")
with open(bkp_name, "w") as bkp:
    bkp.writelines(sent_lines)

```

```

else:
    print("✗ alert.txt not found.")

time.sleep(5)

```

5.9.3 IoT Camera System:

```

from flask import Flask, Response, render_template_string, request, redirect, url_for
from picamera2 import Picamera2
import cv2

# Initialize Flask app
app = Flask(__name__)

# Camera configuration
camera = Picamera2()
camera.configure(camera.create_video_configuration(main={"size": (640, 480)}))
camera.start()

# Authentication credentials
USERNAME = "admin"
PASSWORD = "raspi"

# HTML template for login page
LOGIN_TEMPLATE = """
<!DOCTYPE html>
<html>
<head>
    <title>IoT Camera Login</title>
    <style>
        body {
            background: #1e1e2f;
            color: white;
            font-family: Arial, sans-serif;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
        }
        .login-box {

```

```

    background: #2d2d44;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 0 15px #000;
    width: 300px;
    text-align: center;
}
input[type="text"], input[type="password"] {
    width: 90%;
    padding: 10px;
    margin: 12px 0;
    border: none;
    border-radius: 5px;
}
input[type="submit"] {
    background: #4CAF50;
    color: white;
    border: none;
    padding: 10px 20px;
    border-radius: 5px;
    cursor: pointer;
    margin-top: 10px;
}
input[type="submit"]:hover {
    background: #45a049;
}
.error {
    color: red;
    margin-top: 10px;
    font-size: 0.9em;
}
}
</style>
<script>
document.addEventListener("DOMContentLoaded", function() {
    const inputs = document.querySelectorAll("input");
    inputs.forEach((input, index) => {
        input.addEventListener("keypress", function(e) {
            if (e.key === "Enter") {
                e.preventDefault();
                if (index < inputs.length - 1) {
                    inputs[index + 1].focus();
                } else {
                    document.getElementById("loginForm").submit();
                }
            }
        });
    });
});

```

```

        }
    });
});
});
</script>
</head>
<body>
    <div class="login-box">
        <h2>IoT Camera Login</h2>
        <form method="POST" id="loginForm">
            <input type="text" name="username" placeholder="Username" autofocus
required><br>
            <input type="password" name="password" placeholder="Password" required><br>
            <input type="submit" value="Login">
        </form>
        {% if error %}
        <div class="error">{{ error }}</div>
        {% endif %}
    </div>
</body>
</html>
"""

```

```

def generate_frames():
    """Video streaming generator function"""
    while True:
        frame = camera.capture_array()
        frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        _, buffer = cv2.imencode('.jpg', frame)
        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + buffer.tobytes() + b'\r\n')

@app.route('/', methods=['GET', 'POST'])
def login():
    """Login page handler"""
    error = None
    if request.method == 'POST':
        if request.form['username'] == USERNAME and request.form['password'] ==
PASSWORD:
            return redirect(url_for('video_feed'))
        else:
            error = "Invalid username or password"
    return render_template_string(LOGIN_TEMPLATE, error=error)

```

```

@app.route('/video')
def video_feed():
    """Video streaming route"""
    return Response(generate_frames(), mimetype='multipart/x-mixed-replace;
boundary=frame')

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=5000)

```

CHAPTER 6

RESULT

The Snort IDS-based IoT security system effectively detected cyber threats including network scans, brute force attacks, and SQL injections, delivering real-time Telegram alerts within seconds on Raspberry Pi hardware. The system demonstrated reliable threat detection capabilities while maintaining efficient performance, with particular effectiveness in identifying network reconnaissance activities. Its seamless integration of intrusion detection with instant alerting provides robust protection for IoT environments.

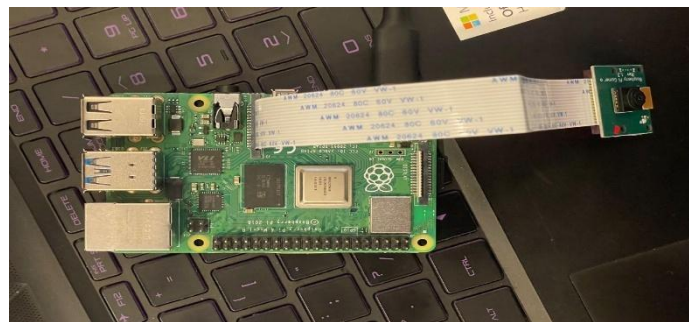


Fig. 6.1 RPi with Camera module

```
memory scale: KB
total memory: 85.8652
pattern memory: 20.3926
match list memory: 30.2422
transition memory: 33.8555
fast pattern only: 1
ppid: MaxRss diff: 3072
ppid: patterns loaded: 300
-----
cap DAQ configured to passive.
ommencing packet processing
+ [0] wlan0
4/25-11:01:38.993376 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2530 -> 192.168.1.11:80
4/25-11:01:38.993423 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2531 -> 192.168.1.11:80
4/25-11:01:38.993466 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2532 -> 192.168.1.11:80
4/25-11:01:38.996468 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2533 -> 192.168.1.11:80
4/25-11:01:38.996580 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2534 -> 192.168.1.11:80
4/25-11:01:38.996629 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2535 -> 192.168.1.11:80
4/25-11:01:38.996674 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2536 -> 192.168.1.11:80
4/25-11:01:38.996720 [**] [1:1000001:1] "[SCAN] Nmap SYN Scan Detected" [**] [Priority: 0] {TCP} 192.168.1.13:2537 -> 192.168.1.11:80
```

Fig. 6.2 Snort IDS Log File

```
rashiq@raspberrypi:~/Desktop $ ls
test.py
rashiq@raspberrypi:~/Desktop $ python3 test.py
Monitoring alert.txt...
[✓] Alert sent
[✓] Alert sent
[✓] Alert sent
[✓] Alert sent
[✓] Alert sent
[✓] Alert sent
```

Fig. 6.3 Python telegram alert script

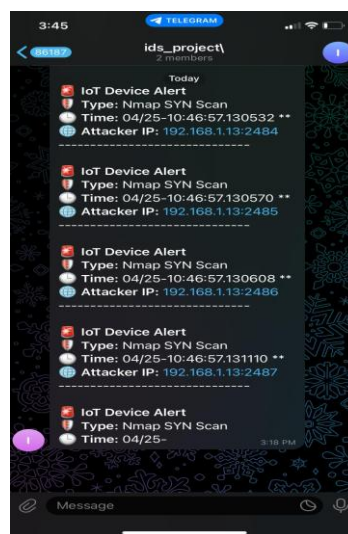


Fig. 6.4 Alert message

CHAPTER 7

CONCLUSION & FUTURE ENHANCEMENT

In conclusion, the development of an integrated IoT security system leveraging the Snort Intrusion Detection System (IDS) on the Raspberry Pi (RPi) platform represents a significant advancement in safeguarding IoT ecosystems against cyber threats. Through the seamless integration of hardware and software components, the system offers robust detection and alerting capabilities to identify and respond to a wide range of cyberattacks targeting IoT devices.

The utilization of the lightweight and cost-effective RPi platform as the host for Snort IDS ensures optimal resource utilization while maintaining robust security protocols. This enables the system to effectively monitor network traffic and detect suspicious activities in real-time, thereby enhancing the overall security posture of IoT environments.

Furthermore, the integration of real-time alerting mechanisms, such as Telegram, empowers administrators to promptly respond to detected threats and take appropriate mitigation actions to safeguard IoT networks. By providing timely notifications and actionable insights, the system enables proactive incident response and minimizes the potential impact of cyber-attacks on IoT devices and infrastructure.

Looking ahead, future enhancements to the system, such as the integration of an Intrusion Prevention System (IPS), enhanced alerting mechanisms, and support for cloud-based security services, hold promise for further strengthening its capabilities and adapting to evolving cybersecurity challenges in IoT environments. Through ongoing research and development efforts, the IoT security system will continue to evolve and remain at the forefront of protecting IoT ecosystems against emerging threats.

In essence, the proposed IoT security system represents a comprehensive and effective solution for mitigating cyber risks in IoT environments, safeguarding critical infrastructure, and ensuring the secure and reliable operation of IoT devices in an increasingly interconnected world.

REFERENCES

•**Sherasiya,S.A.,etal.**

Title: Real-time Anomaly-Based IDS for IoT Networks Using LSTM on RaspberryPi
Published:2025

Summary: This study proposes a real-time anomaly-based IDS for IoT networks using Long Short-Term Memory (LSTM) deep learning algorithms. Implemented on Raspberry Pi, the system achieved high detection accuracy with low false positives. The model was trained on the NSL-KDD dataset, and the system demonstrated adaptability and robustness in smart environments under various attack scenarios such as DoS, R2L, and Probe.

•**Tabrez,S.,etal.**

Title: Rule-Based IDS Using Snort on Raspberry Pi for IoT Environments
Published:2025

Summary: This paper presents a rule-based IDS using Snort on Raspberry Pi 3 for IoT environments. It effectively detected unauthorized access, SYN floods, port scans, and brute-force login attempts. The system included real-time email alerts and a web dashboard for log analysis, empowering administrators to respond promptly to threats. Dynamic IP blocking based on repeated attack attempts was also explored to harden network defenses.

•**Rashid,Q.Y.,etal.**

Title: Hybrid IDS Combining Decision Tree, K-NN, and Random Forest ClassifiersonRaspberryPi
Published:2025

Summary: This research introduces a hybrid IDS combining Decision Tree, K-Nearest Neighbor, and Random Forest classifiers implemented on Raspberry Pi. The ensemble approach improved detection accuracy and reduced false positives compared to single-model systems. A data pre-processing pipeline was also proposed to enhance training efficiency, with real-world traffic simulations to evaluate its generalization across different IoT use cases.

•**Jonnalagadda,S.,etal.**

Title: Smart Home Security Framework Integrating Snort IDS with Raspberry Pi
Published:2024

Summary: This study develops a smart home security framework integrating Snort IDS with Raspberry Pi for real-time intrusion detection. The system captured, filtered, and analyzed network traffic to detect abnormal patterns, generating alerts for activities like port scans and

DoS attempts. Future upgrades proposed include biometric user authentication and cloud-based data backup.

•**Manogna,G.,etal.**

Title: Raspberry Pi-Based IDS Using Decision Tree and K-NN for Anomalous IoT Traffic

Published:2024

Summary: This paper presents a Raspberry Pi-based IDS using Decision Tree and K-Nearest Neighbors to detect anomalous behavior in IoT traffic. Trained on the NSL-KDD dataset, the system was optimized for computation on limited hardware. Testing demonstrated that Raspberry Pi could handle real-time packet processing without significant latency, reducing cloud dependency and enhancing privacy.

•**Kumar,M.,etal.**

Title: Modular and Scalable IDS Using Snort on Raspberry Pi for Educational Institutes

Published:2025

Summary: This research proposes a modular and scalable IDS framework using Snort and Raspberry Pi for securing educational institutes and smart buildings. The system featured layered security with distributed sensors across various departments, custom Snort rules for campus-specific threats, and centralized log aggregation for response from a main server.

•**Jyothirmmai,M.,etal.**

Title: SVM-Based IDS on Raspberry Pi for DoS and R2L Attack Detection in IoT

Published:2024

Summary: This study implements an SVM-based IDS on Raspberry Pi for detecting and mitigating DoS and R2L attacks in IoT networks. A feature reduction technique (PCA) minimized computational load, while GPIO pins triggered alarms upon threat detection. Future work includes extending the system to cover mobile IoT devices and wearable tech.

•**Rizvi,S.,etal.**

Title: Dual-Layer IDS Combining Snort and K-means Clustering on Raspberry Pi

Published:2024

Summary: This paper presents a dual-layer IDS combining Snort with anomaly detection using K-means clustering, deployed on Raspberry Pi. The system monitored traffic patterns, segregated normal from suspicious behavior, and classified unknown threats. It featured a central logging system accessible via a web dashboard and periodic model retraining for adaptive learning, improving zero-day threat detection.

•Asad,M.,etal.

Title: Optimizing Snort Rules for Raspberry Pi with Automated Scripts
Published:2025

Summary: This study focuses on optimizing Snort rules for Raspberry Pi by analyzing rule performance and redundancy. Automated scripts were proposed to evaluate rule conflicts and prioritize high-impact rules. Integrating GPU modules enhanced packet processing speed, with performance benchmarks showing up to a 30% increase in throughput after optimization.

•Debnath,S.,etal.

Title: Distributed IDS Architecture Using Raspberry Pis in a Mesh Network
Published:2024

Summary: This paper designs a distributed IDS architecture using multiple Raspberry Pis in a mesh network. Each Pi handles localized monitoring, while a master node aggregates and visualizes traffic logs. Deployed in a smart office, the system demonstrated high scalability, minimal delay in threat detection, and efficient load distribution, with a fallback mechanism ensuring continuity during node failure.

•Garalov,T.,&El-hajj,M.

Title: Raspberry Pi-Based IDS for Detecting Network Threats Targeting IoT Devices
Published:2024

Summary: This study evaluates the performance of Raspberry Pi-based IDS in detecting network threats targeting IoT devices. The system was tested in a lab simulating real-world smart home scenarios and successfully detected attacks such as ARP spoofing, TCP floods, and unauthorized device access. The research compared it with commercial IDS solutions, showing comparable results at a fraction of the cost.

•Saha,G.

Title:Camera-Based Home IDS Using Raspberry Pi and Firebase
Published:2025

Summary: This paper presents a camera-based home IDS using Raspberry Pi and Firebase. PIR sensors trigger image capture, storing frames in Firebase and sending alerts to a mobile application for cloud-based monitoring. Future enhancements include video analytics and biometric authentication for context-aware intrusion detection.

•Mane,T.,etal.

Title: Wildlife Intrusion Detection System Using Raspberry Pi for Agricultural Fields

Published:2024

Summary: This study designs a wildlife intrusion detection system using Raspberry Pi to protect agricultural fields. Combining thermal imaging, PIR sensors, and animal sound deterrents, AI algorithms processed motion data to identify species and activate deterrents. The system provided real-time SMS alerts and a mobile dashboard for farmers, significantly reducing crop losses.

•B3TA-BLOCKER

Title: LSTM Network-Based IDS on Raspberry Pi for Botnet and Infiltration Detection

Published:2024

Summary: This paper develops a machine learning-powered IDS using LSTM networks deployed on Raspberry Pi. Trained with the CSE-CIC-IDS 2018 dataset, the system identified advanced attacks like botnets and infiltration attempts, achieving over 95% accuracy in lab tests. A secondary feature allowed automatic firewall updates to block IPs of identified attackers.

•Roberts,D.

Title: Snort and Tshark-Based IDS on Raspberry Pi for Educational Training

Published:2024

Summary: This study presents an IDS using Snort and Tshark on Raspberry Pi, with a focus on educational training. The system included an LCD interface for rule management, an email alert system, and a graphical visualization dashboard using Grafana. Custom packet capture filters improved sensitivity to specific threats, demonstrating the viability of DIY IDS solutions for cybersecurity education and small enterprises.

